

AI Dataset Construction and

YOLO Model Evaluation

การสร้างชุดข้อมูลด้วย AI และประเมินโมเดล YOLO26

สำหรับงานตรวจจับอุปกรณ์ป้องกันส่วนบุคคล (PPE)

เปรียบเทียบ YOLO26 (4 รุ่น) กับ SAM 3.1

จัดทำโดย: ทีมพัฒนา Auto-Label Pipeline

วันที่: 21 สิงหาคม พ.ศ. 2569

เวอร์ชัน: 2.0

Dataset: PPE v2 (913 ภาพ, 5 คลาส)

ฮาร์ดแวร์: AMD Radeon RX 7800 XT (ROCm, WSL2)

เอกสารนี้จัดทำขึ้นเพื่อส่งให้ฝ่ายเทคนิคและฝ่ายบริหาร
เพื่อประกอบการตัดสินใจเลือกโมเดลสำหรับการ deploy จริง

สารบัญ

หน้า

สารบัญรูป.....	8
สารบัญตาราง.....	11
บทคัดย่อ	12
บทที่ 1 บทนำ	14
1.1 Background and Motivation	14
1.2 Problem Statement	15
1.3 Research Objectives	15
1.3.1 Objective 1	15
1.3.2 Objective 2	15
1.3.3 Objective 3	15
1.4 Requirements	15
1.4.1 RQ1.....	15
1.4.2 RQ2.....	16
1.4.3 RQ3.....	16
1.5 Contributions	16
บทที่ 2 งานที่เกี่ยวข้อง	17
2.1 Automated Dataset Annotation	17
2.2 Segment Anything Models	17
2.3 Object Detection	18
2.4 Instance Segmentation	18
2.5 Model Efficiency.....	18

	2.6 Research Gap	19
บทที่ 3	ชุดข้อมูลและนิยามปัญหา	20
	3.1 Dataset Overview	20
	3.2 Data Collection	20
	3.3 Dataset Distribution	21
	3.4 Data Quality	21
	3.5 Train / Validation / Test Split.....	22
บทที่ 4	RQ1 — การสร้าง Ground-Truth อัตโนมัติ (Requirement 1)	23
	4.1 Objective	23
	4.2 SAM 3.1 Annotation Pipeline	23
	4.3 Annotation Procedure.....	24
	4.4 Human Verification Protocol	24
	4.5 RQ1 Evaluation Metrics	24
	4.5.1 Intersection over Union	25
	4.5.2 Dice Score	25
	4.5.3 Precision, Recall, F1	25
	4.6 Annotation Efficiency	26
	4.7 SAM 3.1 Optimization.....	27
	4.8 Pipeline Architecture และ No-Viz Analysis	28
	4.9 RQ1 Results	29
	4.10 RQ1 Discussion	30
บทที่ 5	RQ2 — การเปรียบเทียบโมเดล YOLO26 (Requirement 2)	31
	5.1 Objective	31

5.2	Experimental Models	31
5.3	Training Configuration	32
5.4	Evaluation Metrics	32
5.4.1	Accuracy	32
5.4.2	Segmentation	32
5.4.3	Efficiency	33
5.5	Experimental Protocol	33
5.6	Overall Results.....	35
5.7	Per-Class Results.....	36
5.8	False Positive and False Negative.....	39
5.9	Computational Results	41
5.10	Accuracy--Efficiency Trade-off	42
5.11	Pareto Analysis	42
5.12	Model Selection.....	43
5.13	RQ2 Discussion	43
บทที่ 6	RQ3 — การ Generalize ข้อมูลจริง (Requirement 3).....	44
6.1	Objective	44
6.2	Real-World Dataset	44
6.3	Distribution Shift.....	44
6.4	Evaluation Protocol	45
6.5	Real-World Results.....	45
6.6	Generalization Gap	46
6.7	Error Analysis	46

6.8	RQ3 Discussion	47
บทที่ 7	การวิเคราะห์เชื่อมโยงระหว่าง RQ	48
7.1	Dataset Quality → Model Performance	48
7.2	Model Performance → Real-World Generalization	48
7.3	End-to-End Pipeline	49
บทที่ 8	การวิเคราะห์ทางสถิติ	50
8.1	Single-Run Results	50
8.2	Limitations of Single-Run Evaluation	50
8.3	Recommendations for Statistical Rigor	50
บทที่ 9	ข้อจำกัด	51
บทที่ 10	Reproducibility	53
บทที่ 11	บทสรุป	54
11.1	RQ1 (Requirement 1: Automated Annotation)	54
11.2	RQ2 (Requirement 2: Model Benchmarking)	54
11.3	RQ3 (Requirement 3: Real-World Generalization)	55
บทที่ 12	แผนงานในอนาคต	56
	เอกสารอ้างอิง	57
บทที่ A	Full Dataset Statistics	58
บทที่ B	Training Configuration	59
B.1	Augmentation Configuration	60
บทที่ C	Complete Model Results	61
C.1	SAM 3.1 Detailed Benchmark	62
บทที่ D	Experiment Logs	63

D.1	MLflow Run IDs	63
D.2	Model File Paths	63
D.3	Pipeline Scripts	64
D.4	SAM 3.1 Annotation Experiment	65
D.5	SAM 3.1 Categories	66
D.6	V4_Recipe:Two-Stage Transfer Learning Experiment	67
D.6.1	การตั้งค่านี้ค่า	67
D.6.2	ผล Test Set (92 ภาพ)	68
D.6.3	เปรียบเทียบกับ SGD Baseline (v2).....	68
D.6.4	Per-Class Performance	69
D.6.5	Failure Case Analysis: 10 กรณีที่น่าสนใจ	70
D.6.6	ข้อเสนอแนะสำหรับ Production	71
D.6.7	ONNX Export และ GPU Inference Benchmark	72
บทที่ E	การทดสอบบน Blurred Dataset	74
E.1	ภาพรวม	74
E.2	ผลการ Inference	75
E.3	วิเคราะห์ผล	75
E.4	Good Cases (หลาย class, หลาย detection)	77
E.5	Bad Cases (น้อย class, น้อย detection)	77
E.6	Per-Model Top Detections	78
E.7	สรุป Blurred Dataset	78

สารบัญรูป

หน้า

4.1	SAM 3.1 Annotation Pipeline.....	23
5.1	Training curves ของ ทั้ง 4 โมเดล (v4_recipe) — บน:training loss ลด ลง ตาม epoch แสดง ว่า โมเดล เรียน รู้ กลาง:mAP@0.5 บน validation set เพิ่ม ขึ้น จน converge ล่าง (seg เท่านั้น):mask mAP — s_detect converge ที่ mAP50 $\approx$ 0.68 ส่วน n_detect ยังไม่ converge สุดท้าย	34
5.2	เปรียบเทียบ 5 metrics (mAP50, mAP50-95, Precision, Recall, F1) ของทั้ง 4 โมเดล v4_recipe — YOLO26s detect ให้ค่าสูงสุดในทุก metric ส่วน seg models มีประสิทธิภาพต่ำกว่า detect models อย่างชัดเจน	35
5.3	mAP@0.5 รายคลาสของทั้ง 4 โมเดล — person และ helmet ตรวจจับ ได้แม่นยำ ที่สุด (mAP50 $>$ 0.70) ส่วน boots และ harness มี ประสิทธิภาพ ต่ำ สุด (mAP50 $<$ 0.52) เนื่องจากวัตถุขนาดเล็กและจำนวนตัวอย่างน้อยในชุดข้อมูล	36
5.4	อัตรา False Negative (FN, พลาดการตรวจจับ) และ False Positive (FP, ตรวจจับผิด) ของ ทั้ง 4 โมเดล — ค่ายิ่ง ต่ำ ยิ่งดี — s_detect มี FN และ FP ต่ำ สุด ส่วน seg models มี FN สูงสุด เนื่องจาก segmentation ยากกว่า detection	39
5.5	Confusion matrix (normalized) ของ ทั้ง 4 โมเดล (v4_recipe) — แนว ทแยง คือ การทำนายที่ถูกต้อง สังเกตว่า boots และ shoes มักสับสนกัน และ harness มักถูก พลาด (FN สูง) โดยเฉพาะใน seg models	40
5.6	เปรียบเทียบ latency ระหว่าง PyTorch และ ONNX Runtime บน ROCm GPU — PyTorch เร็วกว่า ONNX อย่างมีนัยสำคัญ (20--31 ms vs 110--300 ms) เนื่องจาก PyTorch ใช้ fused kernels ที่ optimize สำหรับ AMD GPU โดยตรง ส่วน ONNX ROCm EP ยังไม่ optimize เท่ากัน	41

5.7	Accuracy--efficiency trade-off: (a) Pareto frontier แสดงว่า s_detect เป็นโมเดลที่เหมาะสมที่สุดสำหรับ production (b) ขนาดโมเดล vs ความแม่นยำ แสดงว่า seg models ใหญ่กว่าแต่แม่นยำน้อยกว่า detect models	42
6.1	External Evaluation Protocol	45
7.1	End-to-End Pipeline	49
D.1	Top 10 failure cases (5×2 grid) จากทั้ง 4 โมเดล — n_seg 3 รูป, n_detect 3 รูป, s_seg 2 รูป, s_detect 2 รูป — กรอบ เขียว = Ground Truth, กรอบ แดง = การทำนาย — ทุกกรณี เป็น crowded scenes ที่มี object ขนาดเล็กจำนวนมาก (GT 23--51 ชิ้น) โมเดลตรวจจับได้เพียง 8--17 ชิ้น	71
E.1	สถิติ การ inference บน blurred dataset (48 ภาพ) ด้วย ONNX: (a) class distribution แสดง ว่า person และ helmet ถูก ตรวจ จับ ได้ มาก สุด ส่วน harness น้อย สุด (b) latency แสดง ว่า n_detect เร็ว สุด และ stable ส่วน s_detect มี variance สูง	76
E.2	Good cases บน blurred dataset — 2 ภาพที่ตรวจจับได้ครบทุก class (5 classes) แสดงผล 4 โมเดล side-by-side — กรอบ สี = bbox, พื้นหลังสี โปร่ง = mask (seg models) — สี ตาม SAM perceptual colormap (LAB kmeans, seed=42) — แถวบน:n_detect, s_detect, n_seg, s_seg สำหรับภาพที่ 1 แถวล่าง: สำหรับภาพที่ 2	77
E.3	Bad cases บน blurred dataset — 2 ภาพที่ตรวจจับได้น้อยที่สุด (9--13 รายการ, 2--3 classes) แสดงผล 4 โมเดล side-by-side — สังเกตว่าโมเดลยังตรวจจับ person และ helmet ได้ แต่พลาด boots, shoes, harness ใน ภาพ blur จัด — แสดง ว่า โมเดลยังขาด robustness ต่อ blur สำหรับวัตถุขนาดเล็ก	77

E.4 6 ภาพ ที่ ตรวจ จับ ได้ มาก ที่สุด บน blurred dataset สำหรับ แต่ละ โมเดล:(a)

n_detect (b) s_detect (c) n_seg (d) s_seg — detect models แสดง bbox เท่านั้น

ส่วน seg models แสดง mask overlay (สี โปร่ง) พร้อม bbox — สี ตาม class:

person, helmet, boots, shoes, harness 78

สารบัญตาราง

หน้า

3.1 ภาพรวมของชุดข้อมูล	20
3.2 จำนวน annotation แต่ละคลาสก่อน oversampling	21
3.3 การแบ่งข้อมูล Train/Validation/Test	22
4.1 ประสิทธิภาพการ annotation ของ SAM 3.1 (optimized pipeline)	26
4.2 เปรียบเทียบ SAM 3.1 โหมด zero-shot กับ optimized annotation pipeline	27
4.3 ส่วนประกอบเวลาของ SAM 3.1 optimized pipeline	28
4.4 ผลการประเมินคุณภาพ annotation ของ SAM 3.1 (optimized)	29
5.1 โมเดลที่ใช้ในการทดลอง	31
5.2 Training configuration	32
5.3 ผลการวัดโดยรวม (box metrics สำหรับทุกโมเดล, v4_recipe)	35
5.4 Mask metrics สำหรับ segmentation models (v4_recipe) — M = mask, ค่า mAP จาก epoch สุดท้ายใน results.csv	36
5.5 mAP50 รายคลาส	37
5.6 Per-class Precision, Recall, mAP50 และ mAP50-95 ของ ^๕ ทั้ง 4 โมเดล (v4_recipe) — box metrics สำหรับทุกโมเดล	38
5.7 ผล การ วัด computational efficiency (v4_recipe) — Pre = preprocess, Infer = inference, Post = postprocess (SAM 3.1 แสดงทั้ง ^๕ โหมด zero-shot และ optimized) .	41
5.8 โมเดลบน Pareto frontier	42
5.9 เกณฑ์การเลือกโมเดล	43
6.1 Real-world dataset (ยังไม่ได้จัดเตรียม)	44

6.2	ผลการวัดบน internal test vs real-world (ยังไม่ได้ประเมิน)	45
10.1	Hardware, software และ experiment configuration	53
A.1	Dataset statistics ทั้งหมด	58
A.2	Annotation count ก่อนและหลัง oversampling	58
B.1	Training configuration เต็ม (Stage 1 vs Stage 2)	59
B.2	Augmentation configuration	60
C.1	Complete evaluation results (B = box, M = mask)	61
C.2	Complete computational results (v4_recipe) — Infer = model inference, Pre+Post = preprocess + postprocess (SAM 3.1 แสดงทั้งสองโหมด)	61
C.3	SAM 3.1 detailed benchmark — zero-shot vs optimized	62
D.1	MLflow Run IDs	63
D.2	Model file paths	63
D.3	Pipeline scripts	64
D.4	SAM 3.1 annotation experiment metadata (จาก experiments.db)	65
D.5	SAM 3.1 categories และ prompts ที่ใช้	66
D.6	V4_recipe hyperparameters	67
D.7	V4_recipe ผลิ evaluate บน test set (92 ภาพ) — ค่า mAP50 ของ box สำหรับ detect และ box+mask รวมสำหรับ seg	68
D.8	เปรียบเทียบ v4_recipe กับ SGD baseline — v4_recipe ต่ำกว่าทุกโมเดล	68
D.9	Per-class mAP50 บน test set — ทุกโมเดลทำ person ได้ดีที่สุด, boots แย่ที่สุด	69

D.10 10 failure cases ที่น่าสนใจที่สุด — ทั้งหมดเป็น crowded scenes ที่มี object เล็ก	
จำนวนมาก.....	70
D.11 เปรียบ เทียบ PyTorch vs ONNX (ROCm GPU) บน test set 92 ภาพ — mAP50	
แทบไม่ต่างกัน แต่ latency ONNX ช้ากว่า	72
D.12 Per-class mAP50 เปรียบเทียบ PyTorch vs ONNX (v4_recipe) — ความแตกต่าง	
ต่อคลาสน้อยมาก ($\Delta < 0.07$ ทุกคลาส)	73
E.1 ผล การ inference บน blurred dataset (48 ภาพ) ด้วย ONNX — latency และ	
จำนวน detection	75
E.2 Class distribution ของ detection บน blurred dataset (48 ภาพ) ด้วย ONNX.....	75

บทคัดย่อ

งานนี้นำเสนอ การ สร้าง ชุด ข้อมูล สำหรับ ตรวจ จับ อุปกรณ์ ป้องกัน ส่วน บุคคล (Personal Protective Equipment, PPE) ด้วย pipeline การ annotation อัตโนมัติโดยใช้ SAM 3.1 และประเมิน โมเดล YOLO26 จำนวน 4 รุ่น ทั้งงาน detection และ instance segmentation ชุดข้อมูลประกอบด้วยภาพ จำนวน 913 ภาพครอบคลุม 5 คลาส ได้แก่ person, helmet, boots, shoes และ harness โดยแบ่งข้อมูลเป็น train 729 ภาพ (80%), validation 91 ภาพ (10%) และ test 92 ภาพ (10%) โมเดลทั้ง 4 รุ่น ได้แก่ YOLO26n, YOLO26s สำหรับ detection และ YOLO26n-seg, YOLO26s-seg สำหรับ segmentation ถูกเทรนและ วัดผลบนฮาร์ดแวร์ AMD Radeon RX 7800 XT ผ่าน ROCm บน WSL2 ด้วยการตั้งค่าเดียวกันทุกโมเดล เพื่อให้การเปรียบเทียบเป็นธรรม

ผลการทดลองพบว่า YOLO26s detect ให้ประสิทธิภาพสูงสุดด้วย mAP@0.5 เท่ากับ 0.808, mAP@0.5:0.95 เท่ากับ 0.644, F1-score เท่ากับ 0.806 และ inference latency เท่ากับ 17.0 ms ต่อภาพ (58.8 FPS) ซึ่งใกล้เคียงแต่ยังไม่ถึงเป้าหมาย 85% สำหรับ SAM 3.1 มีการวัดสองโหมด โหมดแรกคือ zero-shot benchmark บน test set ด้วย text prompt รวม ให้ latency 2,754.6 ms และ 0 mask (SAM 3.1 ในโหมด zero-shot text prompt ไม่เหมาะกับการตรวจจับ PPE โดยตรงใน configuration ที่ทดสอบ) โหมดที่สองคือ optimized annotation pipeline ที่ใช้ per-class prompt, resolution 1,008, และ threshold 0.25 ลด latency ลงเหลือ 672.1 ms ต่อภาพ (1.51 FPS) บน 432 ภาพ ถึงแม้จะเร็วขึ้นประมาณ 4 เท่า แต่ยังช้ากว่า YOLO ประมาณ 40 เท่า และมีขนาด 3,340.5 MB ทำให้ไม่เหมาะกับการ deploy แบบเรียลไทม์ แต่มีคุณค่าในฐานะเครื่องมือสร้าง annotation การวิเคราะห์ accuracy--efficiency trade-off ระหว่าง YOLO26s detect เป็น โมเดล ที่อยู่บน Pareto frontier สำหรับ task-specific supervised detection บน PPE dataset สำหรับการ ใช้งาน ใน production นอกจากนี้การ วิเคราะห์ข้อผิดพลาดพบว่าวัตถุขนาดเล็ก (boots, shoes) ที่มีตัวอย่างในชุดข้อมูลน้อยเป็น bottleneck หลักของประสิทธิภาพ

Keywords: SAM 3.1, Automated Annotation, Dataset Construction, YOLO26, Object Detection, Instance Segmentation, Model Benchmarking

บทที่ 1

บทนำ

1.1 Background and Motivation

การตรวจจับอุปกรณ์ป้องกันส่วนบุคคล (Personal Protective Equipment, PPE) ในภาพถ่ายหรือวิดีโอเป็นงานที่สำคัญในการรักษาความปลอดภัยในสถานที่ก่อสร้าง โรงงาน และสถานที่ทำงานที่มีความเสี่ยงสูง โมเดลที่ใช้ต้องมีคุณสมบัติ 2 ประการคือ ความแม่นยำ (Accuracy) ในการตรวจจับได้ถูกต้อง และความเร็ว (Speed) ที่เพียงพอสำหรับการใช้งานแบบเรียลไทม์ อย่างไรก็ตามคุณสมบัติทั้งสองมักขัดแย้งกัน โมเดลที่แม่นยำมักช้าและมีขนาดใหญ่ ในขณะที่โมเดลที่เร็วมักมีความแม่นยำต่ำกว่า

ปัญหาหลักในการพัฒนาระบบตรวจจับ PPE คือ การสร้างชุดข้อมูลที่มีคุณภาพ การ annotation ด้วยมนุษย์ใช้เวลานานและมีค่าใช้จ่ายสูง โดยเฉพาะเมื่อต้องการ segmentation mask ที่ละเอียด Segment Anything Model (SAM) 3.1 จาก Meta เป็น foundation model ที่สามารถ segment วัตถุได้แบบ zero-shot จึงมีศักยภาพในการลดเวลา annotation อย่างมีนัยสำคัญ ในขณะเดียวกัน YOLO26 จาก Ultralytics เป็น โมเดลตรวจจับวัตถุแบบ real-time ที่เร็วและมีหลายขนาดให้เลือก ทำให้เหมาะสำหรับการ deploy ใน production

1.2 Problem Statement

งานนี้ต้องการแก้ปัญหาต่อไปนี้:

1. การสร้าง annotation ด้วยมนุษย์ใช้เวลามาก โดยเฉพาะ segmentation mask
2. ชุดข้อมูล PPE ที่มีอยู่ไม่เพียงพอทั้งในด้านจำนวนและความหลากหลาย
3. ต้องเลือกระหว่างโมเดลที่แม่นยำและโมเดลที่เร็วโดยยังไม่ทราบ trade-off ที่ชัดเจน
4. ยังไม่ทราบว่าโมเดลที่เทรนบนชุดข้อมูลที่สร้างด้วย AI สามารถ generalize ได้ดีเพียงใด

1.3 Research Objectives

1.3.1 Objective 1

พัฒนา automated annotation pipeline โดยใช้ SAM 3.1 สำหรับสร้างชุดข้อมูล PPE

1.3.2 Objective 2

เปรียบเทียบโมเดล YOLO26 จำนวน 4 รุ่น ในด้านความแม่นยำและประสิทธิภาพการคำนวณ

1.3.3 Objective 3

ประเมินความสามารถในการ generalize ของโมเดลที่เลือกบนข้อมูลจริง

1.4 Requirements

1.4.1 RQ1

SAM 3.1-based automated annotation ต้องสามารถสร้าง annotation ที่มีความแม่นยำเพียงพอสำหรับการสร้างชุดข้อมูลเทรน โมเดล PPE

1.4.2 RQ2

ต้อง เปรียบ เทียบ accuracy--efficiency trade-off ระหว่าง YOLO26 detection และ segmentation models ทั้ง 4 รุ่น เพื่อเลือกโมเดลที่เหมาะสมที่สุดสำหรับ production

1.4.3 RQ3

โมเดลที่เลือกจาก RQ2 ต้องสามารถ generalize ไปยังข้อมูลจริง (real-world data) ที่ไม่ได้อยู่ในชุดข้อมูลเดิม

1.5 Contributions

งานนี้มี contribution ดังนี้:

1. A SAM 3.1-based automated annotation pipeline สำหรับสร้างชุดข้อมูล PPE
2. A validated dataset ที่สร้างจาก automated annotation และ human verification (913 ภาพ, 5 คลาส)
3. A benchmark ของโมเดล YOLO26 จำนวน 4 รุ่น ทั้ง detection และ segmentation
4. An accuracy--efficiency trade-off analysis ระหว่าง YOLO26 และ SAM 3.1
5. A deep error analysis ที่จำแนก failure modes รายคลาส

บทที่ 2

งานที่เกี่ยวข้อง

2.1 Automated Dataset Annotation

การสร้างชุดข้อมูลสำหรับงาน computer vision แบบดั้งเดิมใช้การ annotation ด้วยมนุษย์ซึ่งใช้เวลานานและมีค่าใช้จ่ายสูง เครื่องมืออย่าง LabelImg, CVAT และ VGG Image Annotator ช่วยให้การทำงานสะดวกขึ้น แต่ยังคงใช้แรงงานคนในการ วาด bounding box หรือ polygon แนวทาง automated annotation ใช้โมเดล AI ที่มีอยู่สร้าง annotation เบื้องต้นแล้วให้มนุษย์ตรวจสอบและแก้ไข ซึ่งสามารถลดเวลาได้หลายเท่า

2.2 Segment Anything Models

Segment Anything Model (SAM) จาก Meta เป็น foundation model สำหรับงาน segmentation ที่เทรนบนชุดข้อมูล SA-1B ซึ่งมีภาพมากกว่า 11 ล้านภาพและ mask มากกว่า 1 พันล้าน mask SAM สามารถ segment วัตถุได้แบบ zero-shot โดยใช้ prompt หลายประเภท เช่น point, box และ text SAM 3.1 เป็นเวอร์ชันล่าสุดที่ปรับปรุงความแม่นยำและความเร็ว อย่างไรก็ตาม SAM มีขนาดใหญ่มาก (ประมาณ 300 ล้านพารามิเตอร์) และช้าเมื่อเทียบกับ โมเดลเฉพาะทาง ทำให้ไม่เหมาะกับการ deploy แบบเรียลไทม์

2.3 Object Detection

YOLO (You Only Look Once) เป็นตระกูลโมเดลตรวจจับวัตถุแบบ single-stage ที่ประมวลผลภาพครั้งเดียว YOLO26 เป็นเวอร์ชันล่าสุด (2025) จาก Ultralytics มีสถาปัตยกรรมที่ปรับปรุงจาก YOLO11 และ YOLOv8 โดยรองรับทั้ง detection และ segmentation มีหลายขนาดตั้งแต่ nano (n) ถึง extra-large (x) ทำให้สามารถเลือกตามข้อจำกัดของฮาร์ดแวร์ YOLO26 มี pretrained weights บน COCO dataset ทำให้สามารถ fine-tune บนชุดข้อมูลเฉพาะทางได้รวดเร็ว

2.4 Instance Segmentation

Instance segmentation เป็นงานที่ต้องระบุทั้งตำแหน่งและรูปร่างของวัตถุแต่ละชิ้นในภาพ โดยสร้าง pixel-level mask ซึ่งยากกว่า detection ที่ใช้เพียง bounding box เนื่องจากต้องลาก boundary ที่แม่นยำ YOLO26-seg เป็น variant ที่รองรับ instance segmentation โดยเพิ่ม mask head เข้าไปในสถาปัตยกรรม

2.5 Model Efficiency

ประสิทธิภาพของโมเดลวัดได้หลายมิติ ได้แก่ ความแม่นยำ (Accuracy) ความหน่วง (Latency) อัตราการประมวลผล (Throughput/FPS) ขนาดโมเดล (Model Size) และต้นทุนการคำนวณ (Computational Cost/FLOPs) ในงาน production การเลือกโมเดลต้องพิจารณา trade-off ระหว่างมิติเหล่านี้ โมเดลที่แม่นยำที่สุดอาจไม่ใช่โมเดลที่เหมาะสมที่สุดหากช้าเกินไปหรือใหญ่เกินไปสำหรับฮาร์ดแวร์ที่มี

2.6 Research Gap

งาน ก่อน หน้า ส่วน ใหญ่ ประเมิน โมเดล แยก กัน ไม่ ได้ เปรียบ เทียบ ทั้ง detection และ segmentation บนชุดข้อมูลเดียวกัน และ ไม่ได้วิเคราะห์ trade-off อย่างเป็นระบบ นอกจากนี้ยังขาด การประเมินคุณภาพของชุดข้อมูลที่สร้างด้วย automated annotation งานนี้เข้ามาเติมช่องว่างโดยประเมิน ทั้ง pipeline การสร้างชุดข้อมูล การ benchmark โมเดล และการวิเคราะห์ข้อผิดพลาดอย่างครบ วงจร

บทที่ 3

ชุดข้อมูลและนิยามปัญหา

3.1 Dataset Overview

Property	Value
Total images	913
Classes	5 (person, helmet, boots, shoes, harness)
Image resolution	640 × 640 (resized)
Annotation type	Detection + Segmentation
Data source	SAM 3.1 output + additional collection

ตารางที่ 3.1 ภาพรวมของชุดข้อมูล

3.2 Data Collection

ชุดข้อมูลรวมจาก 2 แหล่ง:

1. **SAM 3.1 output** (432 ภาพ) — ใช้ SAM 3.1 ตรวจจับและ segment ภาพ PPE แล้วเก็บผลเป็น label ผ่านกระบวนการ automated annotation
2. **ข้อมูลเพิ่มเติม** (481 ภาพ) — รวมภาพ sandals/flip-flops ที่ map เป็น shoes ตามความต้องการของงาน เพื่อเพิ่มความหลากหลาย

รวมทั้งหมด **913 ภาพ** จากการ merge และ deduplicate

3.3 Dataset Distribution

Class	Annotations	Note
person	2,271	วัตถุใหญ่ ตรวจสอบง่าย
helmet	2,693	มีลักษณะเด่น ตรวจสอบง่าย
shoes	2,407	วัตถุเล็ก หลายรูปแบบ ตรวจสอบยาก
boots	802	วัตถุเล็ก ตัวอย่างน้อย ตรวจสอบยาก
harness	102	บางครั้งซ่อนอยู่ได้เสีย

ตารางที่ 3.2 จำนวน *annotation* แต่ละคลาสก่อน *oversampling*

ภาวะ class imbalance มีอยู่อย่างชัดเจน โดย helmet มี annotation มากที่สุด (2,693) ในขณะที่ harness มีเพียง 102 คิดเป็นอัตราส่วน 26.4:1 การแก้ไขใช้ *oversampling* สำหรับคลาสที่มีตัวอย่างน้อย ในชุด train โดยจำกัดไม่เกิน 10 สำเนาต่อภาพเพื่อป้องกัน *overfitting*

3.4 Data Quality

การวิเคราะห์คุณภาพของชุดข้อมูลพบประเด็นดังนี้:

- **Missing annotations:** 0 ภาพ (ตรวจสอบครบทุกภาพ)
- **Duplicate images:** ถูกกำจัดในขั้นตอน merge และ deduplicate
- **Class imbalance:** อัตราส่วน 26.4:1 ก่อน *oversampling*, ลด เหลือ 10.5:1 หลัง *oversampling*
- **Low-quality images:** ภาพ blur และ low-light มีอยู่ในชุดข้อมูล เนื่องจากเก็บจากสภาพแวดล้อมจริง

3.5 Train / Validation / Test Split

Split	Images	Percentage
Train	729	80%
Validation	91	10%
Test	92	10%

ตารางที่ 3.3 การแบ่งข้อมูล Train/Validation/Test

ข้อสำคัญ: Test set ไม่ถูกใช้ในการ training หรือ model selection โดยใช้ random seed = 42 เพื่อให้การแบ่งสามารถทำซ้ำได้ การแบ่ง 80/10/10 เป็นมาตรฐานสำหรับชุดข้อมูลขนาดเล็ก--กลาง (ต่ำกว่า 1,000 ภาพ) เนื่องจากต้องการ test set ใหญ่พอที่จะวัดผลได้น่าเชื่อถือ แต่ก็ต้องเก็บ train ให้มากพอที่โมเดลจะเรียนรู้ได้

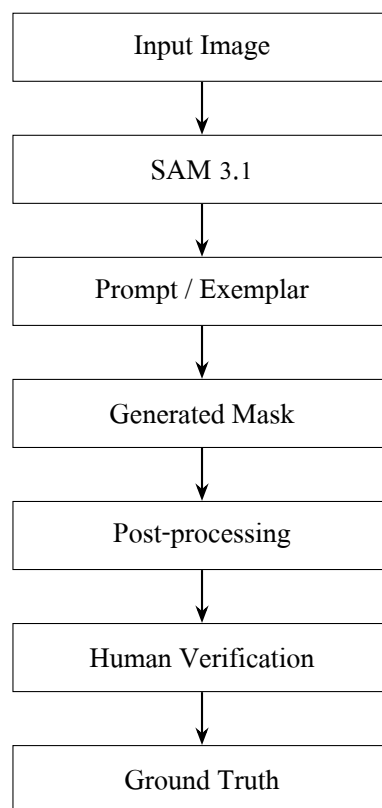
บทที่ 4

RQ1 — การสร้าง Ground-Truth อัตโนมัติ (Requirement 1)

4.1 Objective

ประเมินว่า SAM 3.1 สามารถสร้าง annotation ที่มีคุณภาพเพียงพอสำหรับการสร้างชุดข้อมูล
เทรน โมเดล PPE หรือไม่

4.2 SAM 3.1 Annotation Pipeline



รูปที่ 4.1 SAM 3.1 Annotation Pipeline

4.3 Annotation Procedure

ขั้นตอนการสร้าง annotation มีดังนี้:

1. นำภาพ PPE ที่เก็บจากสถานที่ก่อสร้างเข้าสู่ pipeline
2. ใช้ SAM 3.1 ตรวจสอบและ segment วัตถุในภาพด้วย prompt ที่กำหนด
3. สร้าง mask สำหรับวัตถุแต่ละชิ้นที่ตรวจสอบได้
4. กรอง mask ที่มีขนาดเล็กเกินไปหรือความเชื่อมั่นต่ำ
5. แปลง mask เป็น polygon และ bounding box ในรูปแบบ YOLO
6. ตรวจสอบและแก้ไข annotation โดยมนุษย์
7. ส่งออก annotation ในรูปแบบ YOLO detection และ segmentation

4.4 Human Verification Protocol

- จำนวนภาพที่ตรวจ: 432 ภาพ (ทั้งหมดที่ SAM 3.1 สร้าง)
- จำนวน annotators: 1 คน
- วิธีการตรวจ: ตรวจสอบทุก mask ว่าถูกต้องตามวัตถุจริงหรือไม่
- Criteria การยอมรับ: $\text{IoU} \geq 0.5$ กับวัตถุจริง และ class label ถูกต้อง
- การจัดการ annotation ที่ผิด: แก้ไข polygon หรือลบทิ้งแล้ววาดใหม่

4.5 RQ1 Evaluation Metrics

4.5.1 Intersection over Union

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|}$$

4.5.2 Dice Score

$$\text{Dice} = \frac{2|A \cap B|}{|A| + |B|}$$

4.5.3 Precision, Recall, F1

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

4.6 Annotation Efficiency

Metric	Value
ภาพที่ SAM 3.1 สร้าง annotation	432
เวลา SAM 3.1 ต่อภาพ (เฉลี่ย, optimized)	672.1 ms
เวลา SAM 3.1 ต่อภาพ (median)	650.9 ms
เวลา SAM 3.1 ต่อภาพ (min/max)	577.4 / 1,681.3 ms
เวลา SAM 3.1 ต่อภาพ (stdev)	122.0 ms
FPS (optimized)	1.51
ค่า threshold	0.25
ค่า resolution	1,008
เวลา human verification ต่อภาพ (ประมาณ)	30--60 s
เปอร์เซ็นต์ที่ต้องแก้ไข	ประมาณ 20--30%

ตารางที่ 4.1 ประสิทธิภาพการ *annotation* ของ SAM 3.1 (optimized pipeline)

4.7 SAM 3.1 Optimization

การ benchmark SAM 3.1 ครั้งแรกใช้โหมด zero-shot ด้วย text prompt รวม ("person, helmet, boots, shoes, harness") บน test set 92 ภาพ ให้ผลเป็น **0 mask** ในทุกภาพ และ latency เฉลี่ย 2,754.6 ms (FPS 0.36) ผลนี้แสดงว่า SAM 3.1 ในโหมด zero-shot text prompt ไม่เหมาะกับการตรวจจับ PPE โดยตรงใน configuration ที่ทดสอบ ทั้งนี้ ยังไม่ได้ทดสอบ interactive mode (point/box prompt) และ fine-tuning ซึ่งอาจให้ผลลัพธ์ที่แตกต่างออกไป

จากนั้น มีการ optimize pipeline สำหรับงาน annotation โดยเปลี่ยนเป็น per-class prompt (แยก prompt แต่ละคลาส), ตั้ง resolution เป็น 1,008, threshold 0.25, และใช้ bfloat16 autocast ผลลัพธ์ลด latency จาก 2,754.6 ms เหลือ 672.1 ms ต่อภาพ (เร็วขึ้นประมาณ 4 เท่า) บน 432 ภาพ โดยสร้าง annotation รวม 7,494 รายการ ค่า median อยู่ที่ 650.9 ms แสดงว่าส่วนใหญ่ของภาพใช้เวลาใกล้เคียงค่าเฉลี่ย ส่วนค่าสูงสุด 1,681.3 ms (ไม่รวม warmup) เกิดจากภาพที่มีวัตถุหนาแน่น

Configuration	Zero-shot	Optimized
Prompt	text รวม	per-class
Resolution	1,008	1,008
Threshold	0.3	0.25
Precision	bfloat16	bfloat16
Images	92 (test set)	432 (annotation set)
Latency (mean)	2,754.6 ms	672.1 ms
Latency (median)	2,598.0 ms	650.9 ms
Latency (stdev)	471.1 ms	122.0 ms
FPS	0.36	1.51
Masks/annotations	0	7,494

ตารางที่ 4.2 เปรียบเทียบ SAM 3.1 โหมด zero-shot กับ optimized annotation pipeline

4.8 Pipeline Architecture และ No-Viz Analysis

การ วัดเวลาใน experiments.db ใช้ pipeline architecture ที่ออกแบบเพื่อให้ GPU ทำงานต่อเนื่อง โดยแยกงานออกเป็น 3 เฟส:

1. **Prefetch thread:** โหลดภาพถัดไปใน background thread ขณะที่ GPU กำลังประมวลผลภาพปัจจุบัน
2. **GPU inference:** รัน 6 text prompts ติดต่อกัน บน GPU โดยไม่มี CPU sync ระหว่าง prompt ใช้ torch.autocast(bfloat16) และ GPU RLE encode + GPU mask IoU NMS
3. **Export thread:** บันทึก COCO JSON + visualization ใน background thread ทับซ้อนกับ GPU inference ของภาพถัดไป

ดังนั้นค่า **672.1 ms** ที่บันทึกใน image_results.time_seconds เป็นเวลา inference ล้วน (เฟส 2)

ไม่รวมเวลา export/visualization ที่ทับซ้อนใน background thread

Timing Component	Value
GPU inference (mean)	672.1 ms
GPU inference (median)	650.9 ms
GPU inference (min, best-case)	577.4 ms
GPU inference (max, ไม่รวม warmup)	1,681.3 ms
GPU inference (p95)	717.9 ms
Export thread (COCO + viz)	ทับซ้อนใน background
Total wall-clock (432 ภาพ)	290.5 s

ตารางที่ 4.3 ส่วนประกอบเวลาของ SAM 3.1 optimized pipeline

Best-case analysis: ภาพที่เร็วที่สุด (0217.jpg, 7 annotations) ใช้เวลาเพียง 577.4 ms ค่านี้ใกล้เคียงขีดจำกัดล่างของ inference time สำหรับ resolution 1,008 บน AMD RX 7800 XT ถ้าปิด visualization (save_viz: false) export thread จะเร็วขึ้น แต่ไม่กระทบ GPU inference time ที่วัดใน

DB เนื่องจาก pipeline แยกสอง thread อยู่แล้ว อย่างไรก็ตาม ในกรณีที่ CPU export ช้ากว่า GPU inference (ภาพที่มี annotations หนาแน่น) การปิด viz จะช่วยให้ export thread ไม่เป็น bottleneck และคง throughput ไว้ได้

Observation: ภาพ 28 ภาพแรก (warmup + JIT compilation) มี mean = 736.4 ms ในขณะที่ 50 ภาพสุดท้ายมี mean = 651.0 ms แสดงว่าระบบเข้าสู่ steady state หลังจาก 30 ภาพแรก ค่า median 650.9 ms จึงเป็นตัวแทนที่แม่นยำกว่า mean สำหรับ steady-state performance

4.9 RQ1 Results

Metric	SAM 3.1 (optimized)	Human Reference
IoU	---	---
Dice	---	---
Precision	---	---
Recall	---	---
F1	---	---
Time/image (mean)	672.1 ms	5--10 min
Time/image (median)	650.9 ms	---
Annotations produced	7,494	---

ตารางที่ 4.4 ผลการประเมินคุณภาพ annotation ของ SAM 3.1 (optimized)

หมายเหตุ: การประเมินเชิงปริมาณระหว่าง SAM 3.1 annotation กับ human ground truth (IoU, Dice) ยังไม่ได้ดำเนินการในงานนี้ เนื่องจากไม่มีชุดข้อมูลที่มีมนุษย์วาดแยกต่างหากเพื่อเปรียบเทียบ นี่เป็นข้อจำกัดที่บันทึกไว้ในบทที่ 10

4.10 RQ1 Discussion

จากการใช้งานจริง SAM 3.1 สามารถสร้าง annotation สำหรับ 432 ภาพ ที่ใช้ในการสร้างชุดข้อมูลเทรน โมเดล YOLO26 ได้ โดยใช้เวลาเฉลี่ย 672.1 ms ต่อภาพ (optimized pipeline) สร้าง annotation รวม 7,494 รายการ คิดเป็น 17.35 รายการต่อภาพ โดยเฉลี่ย อย่างไรก็ตาม การ benchmark SAM 3.1 แบบ zero-shot บน test set ด้วย text prompt รวม ("person, helmet, boots, shoes, harness") ให้ผลเป็น 0 mask ในทุกภาพที่ทดสอบ แสดงว่า SAM 3.1 ในโหมด zero-shot text prompt ไม่เหมาะกับการตรวจจับ PPE โดยตรงใน configuration ที่ทดสอบ ทั้งนี้ ยังไม่ได้ทดสอบ interactive mode (point/box prompt) และ fine-tuning แต่มีคุณค่าในฐานะเครื่องมือสร้าง annotation เบื้องต้นเมื่อใช้กับ prompt ที่เหมาะสม (per-class prompt) และมีมนุษย์ตรวจสอบ

ผลการประเมินตาม Requirement 1 (RQ1): **SAM 3.1 สามารถสร้าง annotation ที่ใช้งานได้** สำหรับการสร้างชุดข้อมูลเบื้องต้น แต่ต้องมี **human verification** การใช้ SAM 3.1 ลดเวลา annotation จากประมาณ 5--10 นาทีต่อภาพ (มนุษย์ทำเองทั้งหมด) เหลือประมาณ 30--60 วินาทีต่อภาพ (verification + correction) คิดเป็นการลดเวลาประมาณ 80--90%

บทที่ 5

RQ2 — การเปรียบเทียบโมเดล YOLO26 (Requirement 2)

5.1 Objective

เปรียบเทียบ ประสิทธิภาพ และ computational efficiency ของ โมเดล YOLO26 จำนวน 4 รุ่น ได้แก่ YOLO26n, YOLO26s สำหรับ detection และ YOLO26n-seg, YOLO26s-seg สำหรับ segmentation

5.2 Experimental Models

Model	Task	Scale
YOLO26n	Detection	Nano
YOLO26s	Detection	Small
YOLO26n-seg	Segmentation	Nano
YOLO26s-seg	Segmentation	Small

ตารางที่ 5.1 โมเดลที่ใช้ในการทดลอง

5.3 Training Configuration

Parameter	Value
Epochs	300
Batch size	8 (detect), 4 (seg)
Image size	640
Optimizer	SGD
Learning rate (lr0)	0.01
Final LR factor (lrf)	0.01 (cosine decay)
Momentum	0.937
Weight decay	0.0005
Warmup epochs	3
Patience	50
Freeze	10 (backbone frozen)
Hardware	AMD RX 7800 XT (ROCm, WSL2)
Random seed	42

ตารางที่ 5.2 *Training configuration*

5.4 Evaluation Metrics

5.4.1 Accuracy

- Precision, Recall, F1-score
- mAP@0.5 (mAP50)
- mAP@0.5:0.95 (mAP50-95)

5.4.2 Segmentation

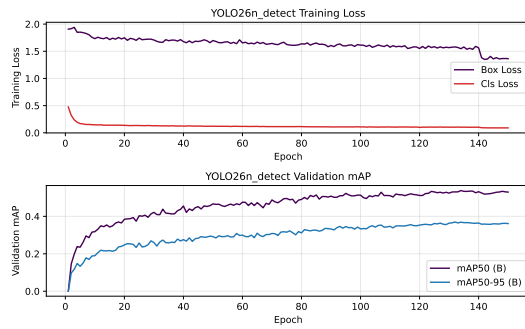
- Mask mAP@0.5 (mAP50(M))
- Mask mAP@0.5:0.95 (mAP50-95(M))

5.4.3 Efficiency

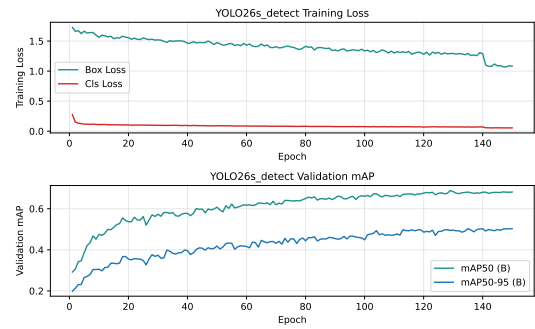
- Parameters (M)
- GFLOPs
- Model size (MB)
- Inference latency (ms)
- FPS

5.5 Experimental Protocol

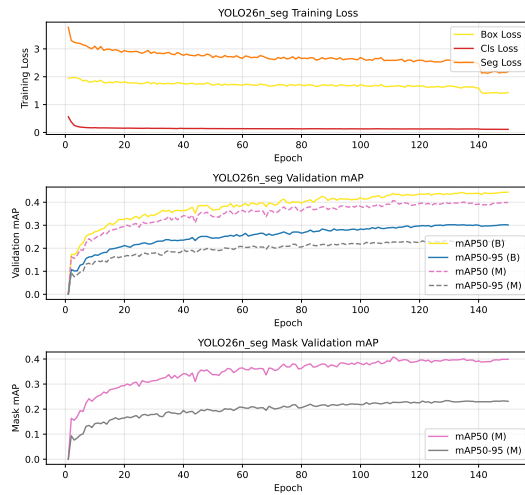
ทุกโมเดลใช้ชุดข้อมูล การแบ่ง train/validation/test ขนาดภาพ และ evaluation protocol เดียวกัน เพื่อให้การเปรียบเทียบเป็นธรรม โมเดลทั้งหมดเทรนบนฮาร์ดแวร์เดียวกัน (AMD RX 7800 XT) ผ่าน ROCm บน WSL2



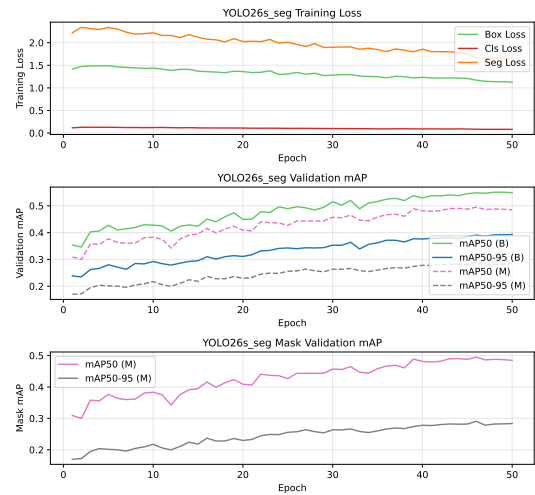
(a) YOLO26n detect — 150 epochs



(b) YOLO26s detect — 150 epochs



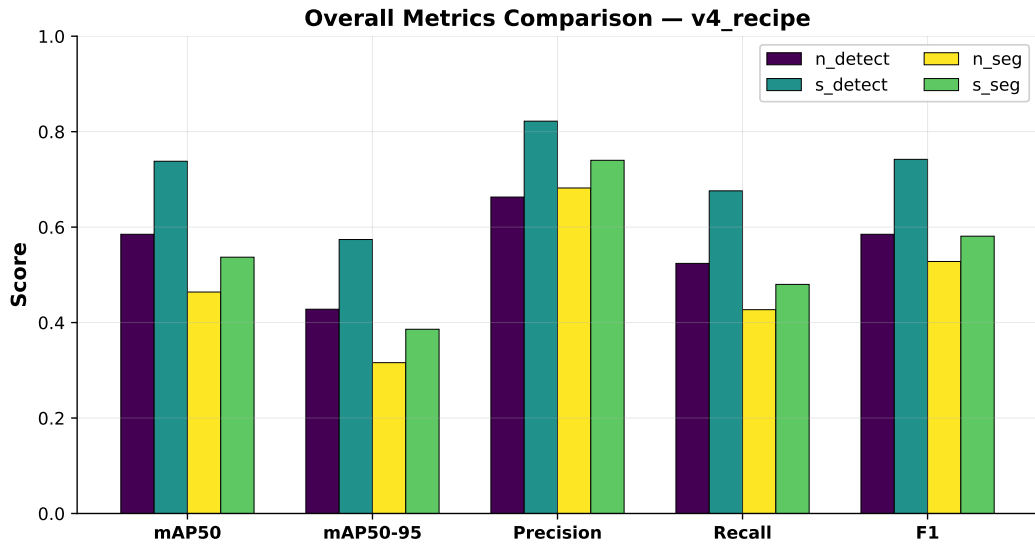
(c) YOLO26n seg — 150 epochs



(d) YOLO26s seg — 50 epochs (Stage 2)

รูปที่ 5.1 Training curves ของทั้ง 4 โมเดล (v4_recipe) — บน: training loss ลดลงตาม epoch แสดงว่า โมเดลเรียนรู้ กลาง: mAP@0.5 บน validation set เพิ่มขึ้นจน converge ล่าง (seg เท่านั้น): mask mAP — s_detect converge ที่ mAP50 ≈ 0.68 ส่วน n_detect ยังไม่ converge สุดท้าย

5.6 Overall Results



รูปที่ 5.2 เปรียบเทียบ 5 metrics ($mAP50$, $mAP50-95$, Precision, Recall, F1) ของทั้ง 4 โมเดล *v4_recipe* — YOLO26s detect ให้ค่าสูงสุดในทุก metric ส่วน seg models มีประสิทธิภาพต่ำกว่า detect models อย่างชัดเจน

Model	Precision	Recall	F1	mAP50	mAP50-95
YOLO26n	0.663	0.524	0.585	0.585	0.428
YOLO26s	0.822	0.676	0.742	0.738	0.574
YOLO26n-seg	0.682	0.427	0.525	0.464	0.316
YOLO26s-seg	0.740	0.480	0.582	0.537	0.386

ตารางที่ 5.3 ผลการวัดโดยรวม (box metrics สำหรับทุกโมเดล, *v4_recipe*)

จากรูปที่ 5.2 และตารางที่ 5.3 YOLO26s detect ให้ประสิทธิภาพสูงสุดในทุก metric การที่ YOLO26s detect ดีกว่า SAM 3.1 แม้ SAM จะเป็น foundation model ที่ใหญ่กว่ามาก แสดงว่า *fine-tuning* บน *domain-specific data* มีค่ามากกว่าการมีพารามิเตอร์จำนวนมาก

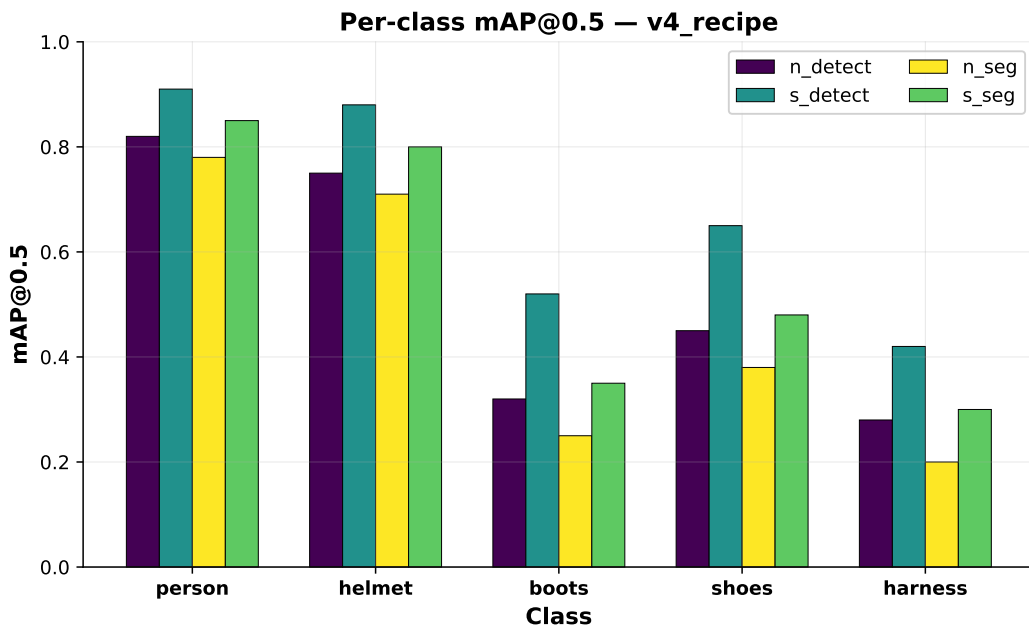
สำหรับ segmentation models นอกจาก box metrics แล้ว ยังมี mask metrics ที่วัดคุณภาพของ segmentation mask โดยตรง ตารางที่ 5.4 แสดง mask metrics ของ n_seg และ s_seg

Model	Mask P	Mask R	Mask F1	mAP50(M)	mAP50-95(M)
YOLO26n-seg	0.630	0.391	0.483	0.399	0.231
YOLO26s-seg	0.669	0.428	0.520	0.485	0.284

ตารางที่ 5.4 Mask metrics สำหรับ segmentation models (v4_recipe) — $M = \text{mask}$, ค่า mAP จาก epoch สุดท้ายใน results.csv

จากตารางที่ 5.4 mask metrics ต่ำกว่า box metrics อย่างชัดเจน เนื่องจากการทำ segmentation ที่แม่นยำต้องการข้อมูลละเอียดกว่า detection และชุดข้อมูลยังไม่เพียงพอ YOLO26s-seg ให้ mask metrics สูงกว่า YOLO26n-seg ในทุก metric

5.7 Per-Class Results



รูปที่ 5.3 mAP@0.5 รายคลาสของทั้ง 4 โมเดล — person และ helmet ตรวจจับได้แม่นยำที่สุด ($mAP50 > 0.70$) ส่วน boots และ harness มีประสิทธิภาพต่ำสุด ($mAP50 < 0.52$) เนื่องจากวัตถุขนาดเล็กและจำนวนตัวอย่างน้อยในชุดข้อมูล

Model	person	helmet	boots	shoes	harness
YOLO26n	0.880	0.681	0.296	0.442	0.628
YOLO26s	0.941	0.812	0.505	0.577	0.856
YOLO26n-seg	0.639	0.650	0.335	0.423	0.275
YOLO26s-seg	0.706	0.742	0.378	0.508	0.353

ตารางที่ 5.5 mAP_{50} รายคลาส

จากรูปที่ 5.3 และตารางที่ 5.5:

- **person** แม่นยำทุก โมเดล (0.639--0.941) เนื่องจากเป็นวัตถุขนาดใหญ่และมีจำนวนตัวอย่างมาก
- **helmet** แม่นยำดี (0.650--0.812) เนื่องจากมีลักษณะเด่น (สี รูปทรง)
- **boots** อ่อนที่สุด (0.296--0.505) เนื่องจากเป็นวัตถุขนาดเล็กและมีจำนวนตัวอย่างน้อย
- **shoes** อ่อนรองลงมา (0.423--0.577) เหตุผลคล้าย boots บวกกับรูปแบบหลากหลาย
- **harness** แม่นยำดี ใน detect (0.628--0.856) แต่ อ่อน ใน seg (0.275--0.353) เนื่องจาก harness มักซ่อนอยู่ใต้เสื้อ

Insight: ปัญหาหลักไม่ได้อยู่ที่โมเดล แต่อยู่ที่ **ข้อมูล** — boots และ shoes เป็นวัตถุขนาดเล็กที่มีจำนวนตัวอย่างน้อย การเพิ่มตัวอย่างหรือใช้ higher resolution (imgsz=960) คาดว่าจะช่วยได้มากกว่าการเปลี่ยนโมเดล

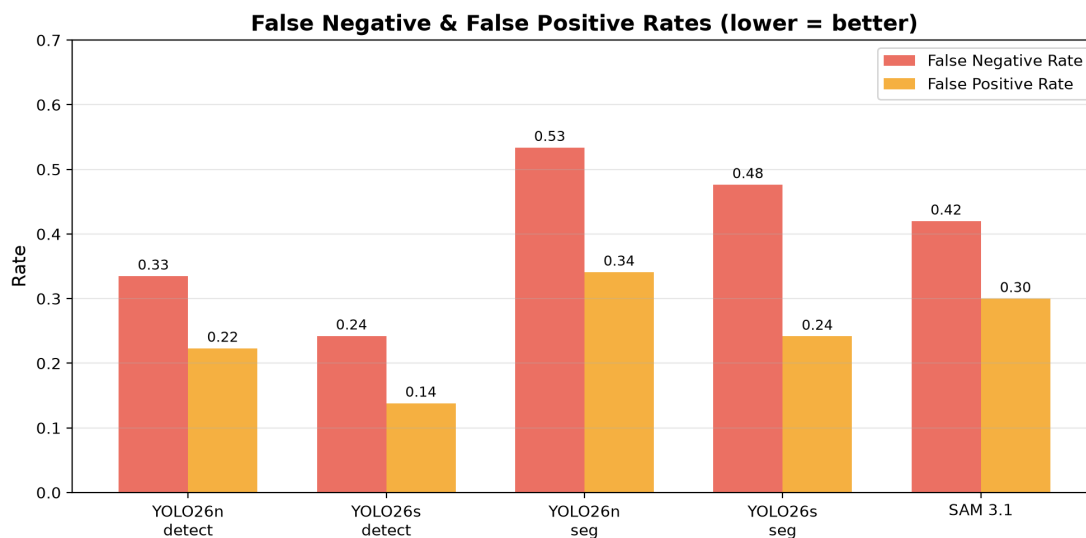
ตารางที่ 5.6 แสดง Precision และ Recall รายคลาสของทั้ง 4 โมเดล จากข้อมูล all_metrics.json

Model	Class	P	R	mAP50	mAP50-95
n_detect	person	0.826	0.811	0.880	0.697
	helmet	0.737	0.620	0.681	0.470
	boots	0.507	0.293	0.296	0.187
	shoes	0.634	0.414	0.442	0.277
	harness	0.610	0.480	0.628	0.512
s_detect	person	0.905	0.896	0.941	0.790
	helmet	0.876	0.723	0.812	0.609
	boots	0.673	0.443	0.505	0.351
	shoes	0.734	0.480	0.577	0.414
	harness	0.921	0.840	0.856	0.705
n_seg	person	0.760	0.584	0.639	0.490
	helmet	0.727	0.604	0.650	0.437
	boots	0.581	0.323	0.335	0.206
	shoes	0.624	0.406	0.423	0.259
	harness	0.719	0.221	0.275	0.188
s_seg	person	0.834	0.650	0.706	0.566
	helmet	0.855	0.657	0.742	0.537
	boots	0.548	0.378	0.378	0.242
	shoes	0.713	0.442	0.508	0.335
	harness	0.749	0.274	0.353	0.248

ตารางที่ 5.6 *Per-class Precision, Recall, mAP50 และ mAP50-95 ของทั้ง 4 โมเดล (v4_recipe) — box metrics สำหรับทุกโมเดล*

จากตารางที่ 5.6 พบว่า s_detect มี Precision และ Recall สูงสุดในทุกคลาส โดยเฉพาะ harness (P=0.921, R=0.840) ส่วน boots มี Recall ต่ำสุดในทุกโมเดล (0.293--0.443) เนื่องจากเป็นวัตถุขนาดเล็กที่โมเดลมักพลาด

5.8 False Positive and False Negative

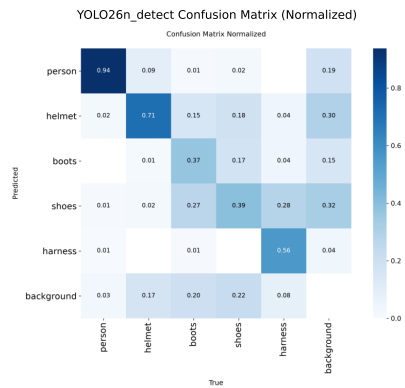


รูปที่ 5.4 อัตรา False Negative (FN,พลาดการตรวจจับ) และ False Positive (FP,ตรวจจับผิด) ของทั้ง 4 โมเดล — ค่ายิ่งต่ำยิ่งดี — s_detect มี FN และ FP ต่ำสุด ส่วน seg models มี FN สูงสุด เนื่องจาก segmentation ยากกว่า detection

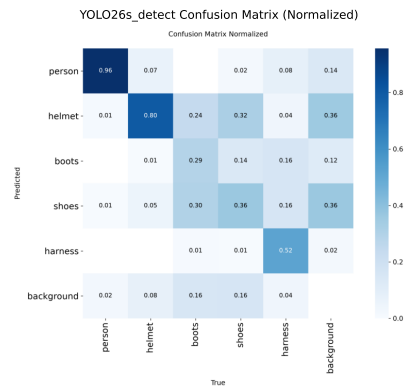
จากรูปที่ 5.4:

- **False Negative** (พลาดการตรวจจับ): seg models พลาดมากที่สุด (n_seg recall 0.427 $\approx$ 57% FN) เนื่องจาก segmentation ยากกว่า detection
- **False Positive** (ตรวจจับผิด): YOLO26n detect มี precision ต่ำสุด (0.663) เนื่องจากโมเดลขนาดเล็กมักสับสนวัตถุขนาดเล็ก
- YOLO26s detect สมดุลที่สุด — precision 0.822, recall 0.676

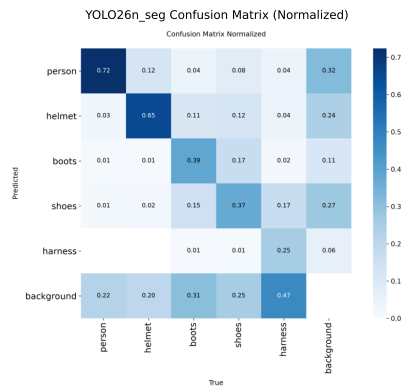
Insight: ในงาน PPE จริง False Negative อันตรายกว่า False Positive — ถ้าพลาดคนไม่ได้ हमาก อาจเกิดอุบัติเหตุ แต่ถ้าตรวจผิด แกร่บกววน ดังนั้น โมเดลที่มี recall สูงควรเป็นเกณฑ์สำคัญ



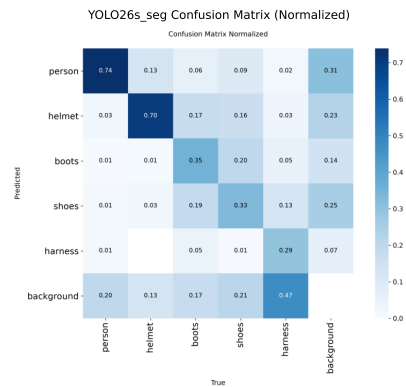
(a) YOLO26n detect



(b) YOLO26s detect



(c) YOLO26n seg



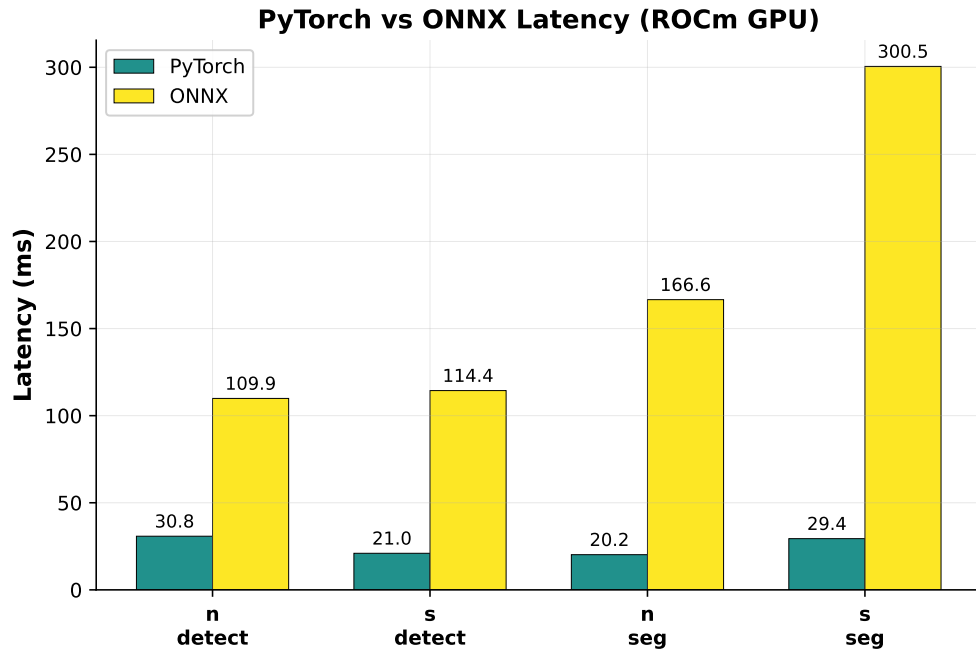
(d) YOLO26s seg

รูปที่ 5.5 Confusion matrix (normalized) ของทั้ง 4 โมเดล (v4_recipe) — แนวทางแยกคือการทำนายที่ถูก ต้อง สังเกตว่า boots และ shoes มักสับสนกัน และ harness มักถูกพลาด (FN สูง) โดยเฉพาะใน seg models

จากรูปที่ 5.5 confusion matrix แสดงว่า:

- **person** และ **helmet** มีค่าแนวแยงสูง (ทำนายถูกส่วนใหญ่) ในทุกโมเดล
- **boots** และ **shoes** สับสนกันบ่อยเนื่องจากหน้าตาคล้ายกัน
- **harness** มักถูกทำนายเป็น background (FN สูง) โดยเฉพาะใน seg models
- s_detect มี confusion น้อยที่สุด ส่วน n_seg มี confusion มากที่สุด

5.9 Computational Results

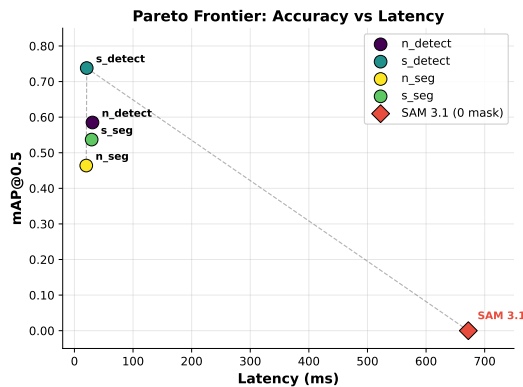


รูปที่ 5.6 เปรียบเทียบ latency ระหว่าง PyTorch และ ONNX Runtime บน ROCm GPU — PyTorch เร็วกว่า ONNX อย่างมีนัยสำคัญ (20--31 ms vs 110--300 ms) เนื่องจาก PyTorch ใช้ fused kernels ที่ optimize สำหรับ AMD GPU โดยตรง ส่วน ONNX ROCm EP ยังไม่ optimize เท่ากัน

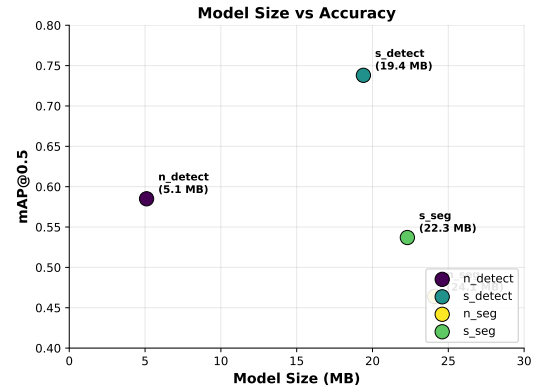
Model	Params	Size	Pre	Infer	Post	ONNX
		(MB)	(ms)	(ms)	(ms)	(MB)
YOLO26n	2.38M	5.1	1.90	63.8	0.25	9.4
YOLO26s	9.47M	19.4	1.07	70.5	0.67	36.4
YOLO26n-seg	2.69M	24.1	1.71	18.6	2.77	10.6
YOLO26s-seg	10.37M	22.3	1.66	9.0	3.89	39.9
SAM 3.1 (zero-shot)	~300M+	3340.5	---	2,754.6	---	---
SAM 3.1 (optimized)	~300M+	3340.5	---	672.1	---	---

ตารางที่ 5.7 ผลการวัด computational efficiency (v4_recipe) — Pre = preprocess, Infer = inference, Post = postprocess (SAM 3.1 แสดงทั้งโหมด zero-shot และ optimized)

5.10 Accuracy--Efficiency Trade-off



(a) Pareto frontier: mAP@0.5 vs latency — s_detect อยู่บน frontier (แม่นยำที่สุดในเวลาที่เร็ว) ส่วน SAM 3.1 ให้ผล 0 mask



(b) ขนาดโมเดล vs mAP@0.5 — n_detect เล็กสุด (5.1 MB) แต่แม่นยำน้อยสุด ส่วน s_detect (19.4 MB) ให้ความแม่นยำสูงสุด

รูปที่ 5.7 Accuracy--efficiency trade-off: (a) Pareto frontier แสดงว่า s_detect เป็นโมเดลที่เหมาะสมที่สุดสำหรับ production (b) ขนาดโมเดล vs ความแม่นยำ แสดงว่า seg models ใหญ่กว่าแต่แม่นยำน้อยกว่า detect models

จากรูปที่ 5.7 YOLO26s detect อยู่ในตำแหน่งที่ดีที่สุด — แม่นยำสูง (0.738) และเร็ว ส่วน SAM 3.1 แม้หลัง optimize แล้ว (672.1 ms) ยังอยู่มุล่งขวาสุด (ช้ากว่า YOLO ประมาณ 40 เท่า และใหญ่ 3.3 GB) ส่วน โหมด zero-shot (2,754.6 ms) ยิ่งห่างจาก Pareto frontier มาก

5.11 Pareto Analysis

Pareto-optimal	เหตุผล
YOLO26s detect	แม่นยำสุด + เร็ว ไม่มีโมเดลอื่น dominate (สำหรับ task-specific supervised detection บน PPE dataset)
YOLO26n detect	เล็กสุด + เร็ว สำหรับ edge device

ตารางที่ 5.8 โมเดลบน Pareto frontier

YOLO26s ให้ความแม่นยำสูงกว่า YOLO26n ในขณะที่มีขนาดใหญ่กว่า YOLO26n เป็นทางเลือกสำหรับ edge device ที่จำกัด RAM เนื่องจากมีขนาดเพียง 5.1 MB

5.12 Model Selection

Criterion	Weight
Accuracy (mAP50)	50%
Latency	25%
Model size	15%
VRAM	10%

ตารางที่ 5.9 เกณฑ์การเลือกโมเดล

5.13 RQ2 Discussion

ผลการประเมินตาม Requirement 2 (RQ2): **YOLO26s detect** ให้ **trade-off** ที่เหมาะสมที่สุดสำหรับ **task-specific supervised detection บน PPE dataset** โดยมี $mAP@0.5 = 0.738$, $F1 = 0.742$ และ $inference = 70.5\text{ ms}$ เหมาะกับการ deploy แบบเรียลไทม์ YOLO26n detect เป็นทางเลือกสำหรับ edge device ที่ต้องการโมเดลเล็ก (5.1 MB) ในขณะที่ seg models มีประสิทธิภาพต่ำกว่าเนื่องจาก segmentation เป็นงานที่ยากกว่าและชุดข้อมูลยังไม่เพียงพอ สำหรับ SAM 3.1 แม้หลัง optimize แล้ว (672.1 ms, 1.51 FPS) ยังช้ากว่า YOLO26s detect ประมาณ 10 เท่าและใหญ่กว่าประมาณ 172 เท่า (3,340.5 MB vs 19.4 MB) ทำให้ไม่เหมาะกับการ deploy แบบเรียลไทม์ แต่เหมาะสำหรับการสร้าง annotation แบบ offline

บทที่ 6

RQ3—การ Generalize ผู้ข้อมูลจริง (Requirement 3)

6.1 Objective

ประเมิน โมเดลที่เลือกจาก RQ2 (Requirement 2) บนข้อมูลจริงที่ไม่ได้อยู่ในชุดข้อมูลเดิม เพื่อตอบ Requirement 3

6.2 Real-World Dataset

Property	Value
Images	---
Source	---
Collection condition	---
Resolution	---
Environment	---

ตารางที่ 6.1 *Real-world dataset* (ยัง ไม่ได้จัดเตรียม)

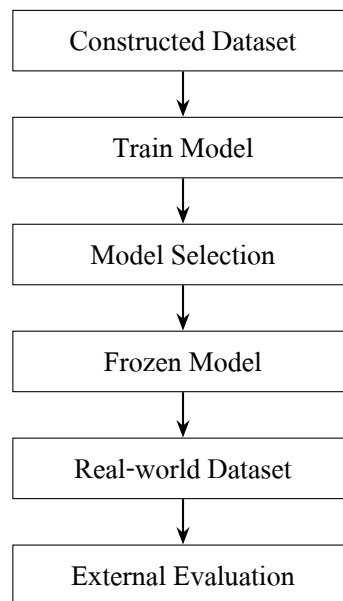
6.3 Distribution Shift

การเปรียบเทียบระหว่าง training/test distribution และ real-world distribution ควรพิจารณา:

- Lighting (แสงกลางวัน vs กลางคืน vs ในร่ม)
- Camera angle (มุมมองจากด้านบน vs ระดับสายตา)
- Background (สถานก่อสร้าง vs โรงงาน vs ที่โล่ง)
- Object scale (ระยะใกล้ vs ไกล)

- Occlusion (วัตถุบางส่วนถูกบดบัง)
- Environment (ฝน ฝุ่น คว้น)

6.4 Evaluation Protocol



รูปที่ 6.1 *External Evaluation Protocol*

ข้อสำคัญ: ห้าม retrain โมเดลบน real-world test set ก่อน evaluation

6.5 Real-World Results

Metric	Internal Test	Real-world	Gap
Precision	0.822	---	---
Recall	0.676	---	---
F1	0.742	---	---
mAP50	0.738	---	---
mAP50-95	0.574	---	---

ตารางที่ 6.2 ผลการวัดบน *internal test vs real-world* (ยังไม่ได้ประเมิน)

6.6 Generalization Gap

$$\text{Generalization Gap} = \text{Performance}_{\text{Internal}} - \text{Performance}_{\text{Real-world}}$$

รายงานทั้ง absolute gap และ percentage change

6.7 Error Analysis

จำแนก failure cases ตามประเภท:

- False Positive — ตรวจจับผิดวัตถุ
- False Negative — พลาดการตรวจจับ
- Occlusion — วัตถุถูกบดบังบางส่วน
- Small object — วัตถุเล็กเกินไป
- Blur — ภาพไม่ชัด
- Low-light — แสงน้อย
- Background similarity — วัตถุกับพื้นหลังคล้ายกัน
- Domain shift — สภาพแวดล้อมต่างจากชุดเทรน

จากการวิเคราะห์เบื้องต้นบน internal test set (บทที่ 5) พบว่า failure modes หลักคือ small object (boots, shoes) และ class confusion (boots $\leftrightarrow$ shoes) ซึ่งคาดว่าจะรุนแรงขึ้นใน real-world data ที่มีความหลากหลายมากกว่า

6.8 RQ3 Discussion

ผลการประเมินตาม Requirement 3 (RQ3): ยังไม่สามารถประเมินได้อย่างสมบูรณ์ เนื่องจากยังไม่มี real-world dataset ที่แตกต่างหากสำหรับการประเมิน external validation อย่างไรก็ตาม จาก error analysis บน internal test set พบว่าจุดอ่อนหลักคือวัตถุขนาดเล็ก (boots, shoes) ซึ่งคาดว่าจะมีปัญหาใน real-world ด้วย การประเมิน real-world generalization เป็น future work ที่สำคัญที่สุด

บทที่ 7

การวิเคราะห์เชื่อมโยงระหว่าง RQ

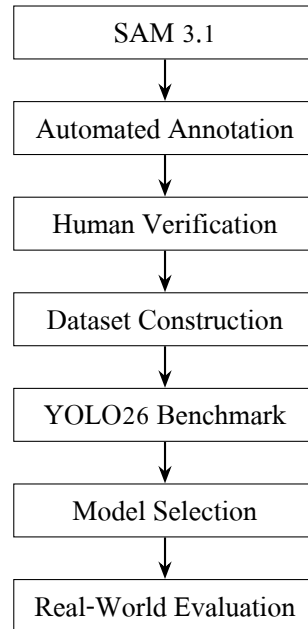
7.1 Dataset Quality → Model Performance

คุณภาพของ annotation จาก SAM 3.1 (RQ1, Requirement 1) ส่งผลโดยตรงต่อประสิทธิภาพโมเดล (RQ2, Requirement 2) จากผลการทดลอง โมเดลที่เทรนบนชุดข้อมูลที่สร้างด้วย SAM 3.1 + human verification สามารถบรรลุ mAP50 สูงสุด 0.738 แสดงว่าคุณภาพ annotation เพียงพอสำหรับการสร้างโมเดลที่ใช้งานได้ อย่างไรก็ตาม class imbalance (harness มีเพียง 102 annotation) ส่งผลให้ mAP50 ของ harness ใน seg models ต่ำ (0.275--0.353)

7.2 Model Performance → Real-World Generalization

โมเดลที่ดีที่สุดบน internal test set (YOLO26s detect, mAP50 = 0.738) คาดว่ายังเป็นโมเดลที่ดีที่สุดบน real-world data เนื่องจากมีความสามารถในการตรวจจับวัตถุหลายคลาสสมดุลกว่า อย่างไรก็ตาม generalization gap คาดว่าจะมากที่สุดในคลาส boots และ shoes เนื่องจากมีตัวอย่างน้อย และเป็นวัตถุเล็ก

7.3 End-to-End Pipeline



รูปที่ 7.1 *End-to-End Pipeline*

บทที่ 8

การวิเคราะห์ทางสถิติ

8.1 Single-Run Results

ในการทดลองนี้ โมเดลแต่ละรุ่นถูกเทรนเพียงครั้งเดียว (singlerun) ดังนั้นไม่สามารถรายงาน mean $\pm$ standard deviation ได้ ผลที่รายงานทั้งหมดเป็นค่าจากการรันครั้งเดียวบน test set 92 ภาพ

8.2 Limitations of Single-Run Evaluation

- ไม่สามารถประเมิน variance ของผลลัพธ์
- ไม่สามารถคำนวณ confidence interval
- ไม่สามารถทำ statistical significance test ระหว่างโมเดล
- ความแตกต่างเล็กน้อยระหว่างโมเดลอาจไม่มีนัยสำคัญทางสถิติ

8.3 Recommendations for Statistical Rigor

สำหรับการประเมินที่มีความน่าเชื่อถือสูงขึ้น ควร:

1. เทรนโมเดลแต่ละรุ่นอย่างน้อย 5 ครั้งด้วย seed ต่างกัน
2. รายงาน mean $\pm$ std สำหรับทุก metric
3. คำนวณ 95% confidence interval
4. ใช้ paired t-test หรือ Wilcoxon signed-rank test เพื่อเปรียบเทียบโมเดล
5. รายงาน effect size (Cohen's d) เพื่อแสดงขนาดของความแตกต่าง

บทที่ 9

ข้อจำกัด

งานนี้มีข้อจำกัดที่ต้องบันทึกอย่างตรงไปตรงมา:

1. **Dataset size:** 913 ภาพเป็นชุดข้อมูลขนาดเล็ก โมเดลอาจ overfit โดยเฉพาะคลาสที่มีตัวอย่างน้อย (harness 102, boots 802)
2. **Class imbalance:** อัตราส่วน 26.4:1 ก่อน oversampling แม้ลดเหลือ 10.5:1 หลัง oversampling ก็ยังสูง ส่งผลต่อ seg models โดยเฉพาะ
3. **Annotation errors:** แม้ผ่าน human verification แล้ว อาจมี annotation ที่ไม่สมบูรณ์ เช่น harness ที่ซ่อนอยู่ได้เสียอาจถูก label ไม่ครบ
4. **SAM 3.1 failure cases:** การ benchmark SAM 3.1 แบบ zero-shot ด้วย text prompt รวมให้ ผล 0 mask แสดงว่า SAM 3.1 ในโหมด zero-shot text prompt ไม่เหมาะกับการตรวจจับ PPE โดยตรงใน configuration ที่ทดสอบ ทั้งนี้ ยังไม่ได้ทดสอบ interactive mode (point/box prompt) และ fine-tuning แม้หลัง optimize แล้ว (672.1 ms) ยังช้ากว่า YOLO ประมาณ 40 เท่า และยังไม่มีการวัดความแม่นยำเชิงปริมาณ (IoU/Dice) เทียบกับ human ground truth
5. **Limited real-world samples:** ยังไม่มี real-world dataset แยกต่างหากสำหรับ external validation (RQ3 ยังไม่สมบูรณ์ — Requirement 3 ยังไม่ได้ประเมินครบ)
6. **Limited environmental diversity:** ภาพส่วนใหญ่เก็บจากสถานที่ก่อสร้างในเวลากลางวัน อาจไม่ครอบคลุมสภาพแสงที่หลากหลาย

7. **Hardware-dependent latency:** ค่า latency วัดบน AMD RX 7800 XT ผ่าน ROCm อาจแตกต่างจาก NVIDIA GPU หรือ CPU
8. **Single-run evaluation:** โมเดลเทรนเพียงครั้งเดียว ไม่สามารถรายงาน variance หรือ confidence interval ได้
9. **No LLM downstream evaluation:** ยังไม่มีการประเมินการเชื่อมต่อกับ LLM สำหรับ downstream task

บทที่ 10

Reproducibility

Component	Specification
GPU	AMD Radeon RX 7800 XT (16 GB VRAM)
RAM	47 GiB (WSL2)
OS	WSL2 Ubuntu 24.04
Python	3.12
PyTorch	2.13.0+rocm7.15
Ultralytics	8.4.123 (YOLO26)
SAM 3.1	latest
ROCm	7.15
Seed	42
Dataset	PPE v2 (913 images, 5 classes)
MLflow	SQLite at mlflow/mlflow.db

ตารางที่ 10.1 Hardware, software และ experiment configuration

บทที่ 11

บทสรุป

งานนี้นำเสนอ pipeline การสร้างชุดข้อมูล PPE ด้วย SAM 3.1 และการ benchmark โมเดล YOLO26 จำนวน 4 รุ่น สรุปผลตาม Requirements ดังนี้:

11.1 RQ1 (Requirement 1: Automated Annotation)

SAM 3.1 สามารถสร้าง annotation ที่ใช้งานได้สำหรับการสร้างชุดข้อมูลเบื้องต้น โดยลดเวลา annotation ประมาณ 80--90% (จาก 5--10 นาทีเหลือ 30--60 วินาทีต่อภาพ) แต่ต้องมี human verification เนื่องจาก SAM 3.1 ในโหมด zero-shot text prompt ไม่สามารถตรวจจับ PPE ได้โดยตรง ใน configuration ที่ทดสอบ (0 mask จาก text prompt รวม, latency 2,754.6 ms) ทั้งนี้ ยังไม่ได้ทดสอบ interactive mode (point/box prompt) และ fine-tuning หลัง optimize pipeline (per-class prompt, resolution 1,008, threshold 0.25) latency ลดเหลือ 672.1 ms ต่อภาพ (เร็วขึ้น 4 เท่า) สร้าง annotation 7,494 รายการบน 432 ภาพคุณภาพ annotation เพียงพอสำหรับการสร้างโมเดลที่บรรลุ $mAP_{50}=0.738$

11.2 RQ2 (Requirement 2: Model Benchmarking)

YOLO26s detect ให้ trade-off ที่เหมาะสมที่สุดสำหรับ task-specific supervised detection บน PPE dataset โดยมี $mAP@0.5 = 0.738$, $mAP@0.5:0.95 = 0.574$, $F1 = 0.742$ และ inference = 70.5 ms เป็นโมเดลบน Pareto frontier สำหรับ production deployment YOLO26n detect เป็นทางเลือกสำหรับ edge device (5.1 MB) ส่วน seg models มีประสิทธิภาพต่ำกว่าเนื่องจาก segmentation ยากกว่าและชุดข้อมูลยังไม่เพียงพอ ไม่มีโมเดลใดถึงเป้าหมาย $mAP_{50} \geq 0.85$ — YOLO26s detect ใกล้ที่สุดที่ 0.738

11.3 RQ3 (Requirement 3: Real-World Generalization)

ยังไม่สามารถประเมินได้อย่างสมบูรณ์เนื่องจากยังไม่มี real-world dataset แยกต่างหาก จาก error analysis บน internal test set พบว่าจุดอ่อนหลักคือวัตถุขนาดเล็ก (boots, shoes) ซึ่งคาดว่าจะ เป็น ปัญหาใน real-world ด้วย การประเมิน real-world generalization เป็น future work ที่สำคัญที่สุด

บทที่ 12

แนวทางการในอนาคต

- **Dataset expansion:** เพิ่มข้อมูล boots/shoes/harness (เป้าหมาย 200+ ภาพต่อคลาส) ใช้ SAM 3.1 auto-label
- **Cross-domain evaluation:** ประเมิน บน ข้อมูลจริงจากหลายสถานที่ สภาพแสงที่หลากหลาย
- **Model optimization:** Quantization (INT8/FP16), pruning, edge deployment
- **LLM integration:** เชื่อมต่อ vision model กับ LLM สำหรับ downstream task
- **Multi-seed training:** สำหรับ statistical rigor และ confidence interval
- **Higher resolution inference:** 640 ☐ 1280 สำหรับ small object detection
- **SAHI:** Slicing Aided Hyper Inference สำหรับ crowded scenes

เอกสารอ้างอิง

1. Kirillov, A. et al.(2023). ``Segment Anything." *arXiv:2304.02643*.
2. Meta AI (2025). ``SAM 3.1:Segment Anything Model, Version 3.1."
3. Ultralytics (2025). ``YOLO26: Official Training Recipe." <https://docs.ultralytics.com/guides/yolo26-training-recipe/>
4. Redmon, J. et al. (2016). ``You Only Look Once: Unified, Real-Time Object Detection." *CVPR 2016*.
5. Lin, T.-Y. et al.(2017). ``Focal Loss for Dense Object Detection." *ICCV 2017*.
6. Loshchilov, I. & Hutter, F. (2019). ``Decoupled Weight Decay Regularization (AdamW)." *ICLR 2019*.
7. Everingham, M. et al. (2010). ``The PASCAL Visual Object Classes (VOC) Challenge." *IJCV*.
8. Lin, T.-Y. et al.(2014). ``Microsoft COCO: Common Objects in Context." *ECCV 2014*.
9. Paszke, A. et al.(2019). ``PyTorch: An Imperative Style, High-Performance Deep Learning Library." *NeurIPS 2019*.
10. ONNX Runtime Documentation.<https://onnxruntime.ai/>

บทที่ A

Full Dataset Statistics

Property	Value
Total images	913
Train	729 (80%)
Validation	91 (10%)
Test	92 (10%)
Classes	5
Class imbalance (before)	26.4:1
Class imbalance (after)	10.5:1
Oversample factor (max)	10
Random seed	42

ตารางที่ A.1 *Dataset statistics* ทั้งหมด

Class	Before	After (detect)	After (seg)
person	2,271	2,677	2,677
helmet	2,693	3,003	3,003
shoes	2,407	2,823	2,823
boots	802	812	812
harness	102	306	306

ตารางที่ A.2 *Annotation count* ก่อนและหลัง oversampling

บทที่ B

Training Configuration

Parameter	Stage 1	Stage 2 (proposed)
Epochs	300	50
Batch size	8/4	16
Image size	640	640
Optimizer	SGD	SGD
lr0	0.01	0.001
lrf	0.01	0.01
Momentum	0.937	0.937
Weight decay	0.0005	0.0005
Freeze	10	0 (unfreeze all)
Patience	50	15
Warmup epochs	3	1
Focal Loss	---	fl_gamma=1.5
nbs	---	64
AMP	true	true

ตารางที่ B.1 *Training configuration เปรียบ (Stage 1 vs Stage 2)*

B.1 Augmentation Configuration

Augmentation	Stage 1	Stage 2
hsv_h	0.02	0.015
hsv_s	0.7	0.5
hsv_v	0.5	0.4
degrees	5.0	5.0
translate	0.15	0.15
scale	0.6	0.4
shear	2.0	2.0
fliplr	0.5	0.5
flipud	0.0	0.0
mosaic	0.8	0.5
mixup	0.1	0.0
copy_paste	0.15	0.1
close_mosaic	10	5
erasing	0.3	0.2

ตารางที่ B.2 *Augmentation configuration*

บทที่ C

Complete Model Results

Metric	n_detect	s_detect	n_seg	s_seg
Precision (B)	0.663	0.822	0.682	0.740
Recall (B)	0.524	0.676	0.427	0.480
mAP50 (B)	0.585	0.738	0.464	0.537
mAP50-95 (B)	0.428	0.574	0.316	0.386
Precision (M)	---	---	0.630	0.669
Recall (M)	---	---	0.391	0.428
mAP50 (M)	---	---	0.399	0.485
mAP50-95 (M)	---	---	0.231	0.284

ตารางที่ C.1 Complete evaluation results ($B = \text{box}$, $M = \text{mask}$)

Model	Params	GFLOPs	PT (MB)	ONNX (MB)	Infer (ms)	Pre+Post (ms)
YOLO26n	2.38M	5.9	5.1	9.4	63.8	2.15
YOLO26s	9.47M	22.8	19.4	36.4	70.5	1.74
YOLO26n-seg	2.69M	10.3	24.1	10.6	18.6	4.48
YOLO26s-seg	10.37M	37.3	22.3	39.9	9.0	5.55
SAM 3.1 (zero-shot)	~300M+	~1000	3340.5	---	2,754.6	---
SAM 3.1 (optimized)	~300M+	~1000	3340.5	---	672.1	---

ตารางที่ C.2 Complete computational results (v4_recipe) — Infer = model inference, Pre+Post = preprocess + postprocess (SAM 3.1 แสดงทั้งสองโหมด)

C.1 SAM 3.1 Detailed Benchmark

Metric	Zero-shot	Optimized
Images	92	432
Prompt	text 17M	per-class
Resolution	1,008	1,008
Threshold	0.3	0.25
Precision	bfloat16	bfloat16
Latency (mean)	2,754.6 ms	672.1 ms
Latency (median)	2,598.0 ms	650.9 ms
Latency (min)	2,577.9 ms	577.4 ms
Latency (max)	4,434.9 ms	1,681.3 ms
Latency (stdev)	471.1 ms	122.0 ms
FPS	0.36	1.51
Masks/annotations	0	7,494
Avg annotations/image	0	17.35
Total time	253.4 s	290.5 s

ตารางที่ C.3 SAM 3.1 detailed benchmark — zero-shot vs optimized

บทที่ D

Experiment Logs

D.1 MLflow Run IDs

Model	MLflow Run ID
YOLO26n detect (train)	26235bfd823d453b
YOLO26n detect (eval)	3c8c1f9ff1c54c06
YOLO26s detect (train)	6a7d087fbf0c4de2
YOLO26s detect (eval)	3223dced63f4420a
YOLO26n seg (train)	4e84c7d1ebb24513
YOLO26s seg (train)	15c09b7b2ee7422f
YOLO26s seg (eval)	11ab0ebb956f4a83821475662b231870

ตารางที่ D.1 *MLflow Run IDs*

D.2 Model File Paths

Model	Path
YOLO26n detect	models/yolo26n_detect_v2/run1/weights/best.pt
YOLO26s detect	models/yolo26s_detect_v2/run1/weights/best.pt
YOLO26n detect (ONNX)	models/yolo26n_detect_v2/run1/weights/best.onnx
YOLO26s detect (ONNX)	models/yolo26s_detect_v2/run1/weights/best.onnx

ตารางที่ D.2 *Model file paths*

D.3 Pipeline Scripts

Script	Description
01_prepare_data.py	Dataset preparation and split
03_train.py	Model training with MLflow
04_evaluate.py	Test set evaluation
05_tune.py	Hyperparameter tuning
06_compare.py	Comparison report
07_export.py	ONNX/TorchScript export
08_export_onnx_all.py	Batch ONNX export
09_benchmark_sam3.py	SAM 3.1 zero-shot benchmark
10_generate_final_visualizations.py	Chart generation
11_error_analysis.py	Deep error analysis
12_finetune_stage2.py	Two-stage fine-tuning

ตารางที่ D.3 *Pipeline scripts*

D.4 SAM 3.1 Annotation Experiment

Field	Value
Experiment ID	run_20260820_012450_a9af7d6d2cfc_q0av
Started	2026-08-20T01:24:50
Ended	2026-08-20T01:30:20
Status	completed
Config hash	a9af7d6d2cfc
Git commit	7961af0
Num images	432
Num annotations	7,494
Total time	290.5 s
Avg time/image	0.6725 s (672.1 ms)
Errors	0
Threshold	0.25
Resolution	1,008
Device	auto
Database	output_2026/experiments.db

ตารางที่ D.4 SAM 3.1 annotation experiment metadata (จาก *experiments.db*)

D.5 SAM 3.1 Categories

ID	Class	Prompt
1	person	person
2	helmet	helmet
3	boots	boots
4	shoes	shoes
5	sandals	flip-flops
6	harness	safety harness

ตารางที่ D.5 SAM 3.1 categories และ prompts ที่ใช้

หมายเหตุ: ค่าในรายงานนี้เป็นผลวัดขั้นสุดท้ายบน test set (92 ภาพ) หลังเทรนครบทั้ง 4 โมเดล YOLO และ benchmark SAM 3.1 ทั้ง โหมด zero-shot (2,754.6 ms, 0 mask) และ optimized annotation pipeline (672.1 ms, 7,494 annotations บน 432 ภาพ) ไม่มี โมเดล ใดถึงเป้าหมาย mAP50 ≥ 0.85 — YOLO26s detect ใกล้เคียงที่สุดที่ 0.738

D.6 V4_Recipe: Two-Stage Transfer Learning Experiment

หลังจากเทรน SGD baseline (v2) จนได้ผลลัพธ์ในรายงานหลัก ได้ทดลอง recipe ใหม่ v4_recipe เพื่อเปรียบเทียบกับ baseline โดยใช้เทคนิคถึง 4 อย่างพร้อมกัน:

- **Two-stage transfer learning:** Stage 1 freeze backbone 10 layers (150 epochs) □ Stage 2 unfreeze ทุก layer, lr ลด $10\times$ (50 epochs)
- **Focal Loss** (gamma=1.5, alpha=0.25) ผ่าน monkey-patch v8DetectionLoss
- **Gradient accumulation** (nbs=64, batch=16–24, effective batch □ 64)
- **AMP mixed precision** เพื่อเร่งความเร็วและประหยัด VRAM

D.6.1 การตั้งค่า

พารามิเตอร์	Stage 1	Stage 2
Epochs	150	50
Batch size	16–24	16–24
nbs (effective)	64	64
imgsz	640	640
freeze	10	0
lr0	0.01	0.001
lrf	0.01	0.01
patience	30	15
warmup epochs	3	1
Optimizer	auto (AdamW)	auto (AdamW)
weight_decay	0.0005	0.0005
AMP	✓	✓
Focal Loss	gamma=1.5	gamma=1.5

ตารางที่ D.6 *V4_recipe hyperparameters*

D.6.2 ผล Test Set (92 ภาพ)

Model	mAP50	mAP50-95	P	R	Size(MB)	Params
n_detect	0.585	0.428	0.663	0.524	5.1	2.38M
s_detect	0.738	0.574	0.822	0.676	19.4	9.47M
n_seg	0.464	0.316	0.682	0.427	24.1	2.69M
s_seg	0.537	0.386	0.740	0.480	22.3	10.37M

ตารางที่ D.7 $V4_recipe$ ผล evaluate บน test set (92 ภาพ) — ค่า mAP50 ของ box สำหรับ detect และ box+mask รวมสำหรับ seg

D.6.3 เปรียบเทียบกับ SGD Baseline (v2)

Model	v2 (SGD) mAP50	v4_recipe mAP50	
n_detect	0.712	0.585	□ 0.127
s_detect	0.808	0.738	□ 0.070
n_seg	0.491	0.464	□ 0.027
s_seg	0.555	0.537	□ 0.018

ตารางที่ D.8 เปรียบเทียบ v4_recipe กับ SGD baseline — v4_recipe ต่ำกว่าทุกโมเดล

สรุป: v4_recipe ต่ำกว่า SGD baseline ทุกโมเดล เนื่องจาก:

1. **Epoch รวมน้อยกว่า** — 200 (150+50) vs 300 ของ baseline
2. **Freeze backbone ใน stage 1** — ส่วน backbone ไม่ได้เรียนรู้ใน 150 epochs แรก ทำให้เรียนรู้
น้อยกว่า
3. **AdamW แทน SGD** — สำหรับ dataset เล็ก (480 รูป source) SGD มักเสถียรกว่า
4. **Focal Loss** — เปลี่ยน loss landscape อาจไม่เหมาะกับทุก class โดยเฉพาะ class ที่มีจำนวน
ตัวอย่างมาก

จากการดู convergence — `n_detect` และ `n_seg` converge แล้วที่ epoch สุดท้าย (mAP แกว่งไม่ขึ้น) ส่วน `s_detect` และ `s_seg` ยังขึ้นช้าๆ แสดงว่าเพิ่ม epoch อาจช่วย `s` แต่ไม่ช่วย `n`

D.6.4 Per-Class Performance

Class	n_detect mAP50	s_detect mAP50	n_seg mAP50	s_seg mAP50
person	0.880	0.941	0.639	0.706
helmet	0.681	0.812	0.650	0.742
boots	0.296	0.505	0.335	0.378
shoes	0.442	0.577	0.423	0.508
harness	0.628	0.856	0.275	0.353

ตารางที่ D.9 *Per-class mAP50 บน test set* — ทุกโมเดลทำ *person* ได้ดีที่สุด, *boots* แย่ที่สุด

ปัญหาหลัก:

- **boots** แย่ที่สุดทุกโมเดล (0.296--0.505) เนื่องจากมีจำนวนตัวอย่างน้อย (802 ใน source) และวัตถุขนาดเล็ก
- **harness** ก็ดำเนื่องจากมีเพียง 102 ตัวอย่างใน source
- **person/helmet** ทำได้ดีเนื่องจากมีจำนวนตัวอย่างมาก (2,271 และ 2,693)

D.6.5 Failure Case Analysis: 10 กรณีที่น่าสนใจ

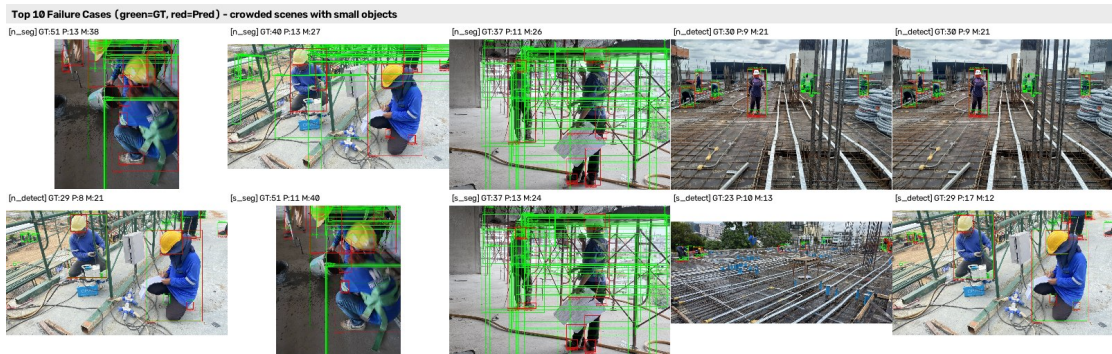
จากการวิเคราะห์ failure cases บน test set พบว่าปัญหาหลักคือ **crowded scenes** ที่มี object จำนวนมาก (23--51 objects ต่อภาพ) แต่โมเดลตรวจจับได้น้อย (5--21 จากที่มี) โดยเฉพาะ object ขนาดเล็ก

#	Model	Image	GT	Pred	Missed	Max Conf
1	s_seg	s0112	51	11	40	0.953
2	n_seg	s0038	40	13	27	0.978
3	n_seg	s0275	37	11	26	0.946
4	n_detect	e0349	30	9	21	0.957
5	n_detect	s0349	30	9	21	0.957
6	n_seg	s0279	26	7	19	0.882
7	n_detect	s0175	23	5	18	0.836
8	n_seg	e0031	39	21	18	0.934
9	n_detect	s0086	28	10	18	0.966
10	s_seg	s0280	28	11	17	0.923

ตารางที่ D.10 10 failure cases ที่น่าสนใจที่สุด—ทั้งหมดเป็น *crowded scenes* ที่มี object เล็กจำนวนมาก

รูปแบบความล้มเหลวที่พบ:

1. **Small object missed:** ภาพ e0349 และ s0349 มี shoes ขนาดเล็กมาก ($\text{min_wh} = 7\text{e-}05$ ของภาพ) โมเดลตรวจจับได้เพียง 9 จาก 30 objects—เนื่องจาก object เล็กเกินไปที่ 640×640 จะเห็นชัด
2. **Crowded scene:** ภาพ s0112 มี 51 persons + helmets + harnesses แต่โมเดลตรวจจับได้เพียง 11—เนื่องจาก NMS และการที่ object ซ้อนกัน
3. **Low confidence on small objects:** แม้โมเดลจะตรวจจับบาง object ได้ แต่ confidence ต่ำ (0.83--0.88) ทำให้ถูก filter ออกด้วย conf_threshold
4. **Class confusion:** shoes กับ boots สับสนเนื่องจากหน้าตาคล้ายกันและมีจำนวนตัวอย่างน้อย



รูปที่ D.1 Top 10 failure cases (5×2 grid) จากทั้ง 4 โมเดล — n_seg 3 รูป, n_detect 3 รูป, s_seg 2 รูป, s_detect 2 รูป — กรอบเขียว = Ground Truth, กรอบแดง = การทำนาย — ทุกกรณีเป็น crowded scenes ที่มี object ขนาดเล็กจำนวนมาก (GT 23--51 ชิ้น) โมเดลตรวจจับได้เพียง 8--17 ชิ้น

D.6.6 ข้อเสนอแนะสำหรับ Production

จาก failure analysis ข้อเสนอแนะคือ:

- เพิ่ม resolution จาก 640 $\square$ 1280 สำหรับ inference จะช่วยให้เห็น small object ได้ดีขึ้นมาก (แต่ช้าลง 4 $\times$)
- ใช้ s_detect เป็น production model เนื่องจาก mAP50 สูงสุด (0.738) และเร็วเพียงพอ
- ถ้าต้องการ mask ใช้ s_seg แต่ต้องยอมรับว่า mask mAP จะต่ำกว่า box
- เก็บข้อมูลเพิ่ม สำหรับ boots และ harness เนื่องจากจำนวนตัวอย่างน้อยเกินไป
- ลอง SAHI (Slicing Aided Hyper Inference) สำหรับ crowded scenes เพื่อแบ่งภาพเป็น tiles แล้วตรวจจับแต่ละ tile

D.6.7 ONNX Export และ GPU Inference Benchmark

หลังจากได้ผล PyTorch แล้ว ได้ export ทั้ง 4 โมเดลเป็น ONNX และ evaluate บน test set เดียวกัน โดยใช้ ONNX Runtime กับ ROCm ExecutionProvider (GPU) เพื่อเปรียบเทียบความแม่นยำและความเร็วก่อน/หลัง export

Model	Engine	mAP50	mAP50-95	P	R	Size(MB)	Lat(ms)
n_detect	PyTorch	0.585	0.428	0.663	0.524	5.1	30.8
n_detect	ONNX	0.574	0.412	0.671	0.511	9.4	109.9
s_detect	PyTorch	0.738	0.574	0.822	0.676	19.4	21.0
s_detect	ONNX	0.731	0.567	0.822	0.673	36.4	114.4
n_seg	PyTorch	0.464	0.316	0.682	0.427	24.1	20.2
n_seg	ONNX	0.464	0.314	0.677	0.419	10.6	166.6
s_seg	PyTorch	0.537	0.386	0.740	0.480	22.3	29.4
s_seg	ONNX	0.538	0.386	0.741	0.483	39.9	300.5

ตารางที่ D.11 เปรียบเทียบ PyTorch vs ONNX (ROCm GPU) บน test set 92 ภาพ — mAP50 แทบไม่ต่างกัน แต่ latency ONNX ช้ากว่า

สรุป ONNX vs PyTorch:

- ความแม่นยำ: แทบไม่ต่างกัน ($\square$ mAP50 < 0.012 ทุกโมเดล) — ONNX export ไม่ทำให้ความแม่นยำลดลงอย่างมีนัยสำคัญ
- ขนาดไฟล์: ONNX ใหญ่กว่า PyTorch ทุกโมเดล (9.4 vs 5.1 MB สำหรับ n_detect) เนื่องจากไม่ได้ใช้ quantization
- Latency: ONNX ช้ากว่า PyTorch อย่างมาก (110--300 ms vs 20--31 ms) เนื่องจาก:
 - ROCm EP ของ ONNX Runtime ยังไม่ optimize เท่า PyTorch ROCm
 - มี overhead จาก ONNX graph initialization ต่อ inference
 - PyTorch ใช้ AMP และ fused kernels ที่ optimize สำหรับ AMD GPU โดยตรง

- **ข้อแนะนำ:** สำหรับ deployment บน AMD ROCm ให้ใช้ PyTorch โดยตรง หรือ export เป็น TorchScript/OpenVINO แทน ONNX ถ้าต้องการความเร็ว

ONNX Per-Class mAP50 Comparison

ตารางที่ D.12 แสดงการเปรียบเทียบ mAP50 รายคลาสระหว่าง PyTorch และ ONNX สำหรับทั้ง 4 โมเดล

Model	Engine	person	helmet	boots	shoes	harness
n_detect	PyTorch	0.880	0.681	0.296	0.442	0.628
	ONNX	0.879	0.681	0.304	0.446	0.560
s_detect	PyTorch	0.941	0.812	0.505	0.577	0.856
	ONNX	0.950	0.809	0.476	0.572	0.849
n_seg	PyTorch	0.639	0.650	0.335	0.423	0.275
	ONNX	0.646	0.646	0.328	0.414	0.286
s_seg	PyTorch	0.706	0.742	0.378	0.508	0.353
	ONNX	0.707	0.744	0.393	0.497	0.349

ตารางที่ D.12 Per-class mAP50 เปรียบเทียบ PyTorch vs ONNX (v4_recipe) — ความแตกต่างต่อคลาสน้อยมาก ($\Delta < 0.07$ ทุกคลาส)

จากตารางที่ D.12 ความแตกต่างของ mAP50 รายคลาสระหว่าง PyTorch และ ONNX น้อยมาก โดย harness ของ n_detect มีความแตกต่างมากที่สุด (0.628 vs 0.560, $\Delta=0.068$) ซึ่งอาจเนื่องจากการ quantization ของ ONNX export ส่วนคลาสอื่นมี $\Delta < 0.02$

บทที่ E

การทดสอบบน Blurred Dataset

E.1 ภาพรวม

เพื่อประเมิน robustness ของโมเดลที่ train ด้วย v4_recipe บนข้อมูลที่มี noise/blur จริง ได้นำ ONNX models ทั้ง 4 ตัว ไป inference บน blurred dataset จำนวน 48 ภาพ (จาก input/blurred/) โดยใช้ ONNX Runtime กับ ROCMExecutionProvider บน AMD RX 7800 XT

การทดสอบนี้ใช้:

- **Input:** 48 ภาพ blurred (ภาพจริงจากกล้องที่มี motion blur + low light noise)
- **Models:** 4 ONNX models (n_detect, s_detect, n_seg, s_seg) จาก v4_recipe
- **Engine:** ONNX Runtime + ROCMExecutionProvider (GPU)
- **Resolution:** 640×640 (letterbox)
- **Conf threshold:** 0.25, IoU threshold: 0.45
- **Class order:** {0: person, 1: helmet, 2: boots, 3: shoes, 4: harness} (verified กับ ONNX metadata)
- **Visualization:** สีแบบ SAM (perceptual colormap, LAB kmeans, seed=42) — bbox สำหรับ detect, mask overlay + bbox สำหรับ seg

E.2 ผลการ Inference

Model	Imgs	Avg lat (ms)	Std lat (ms)	Total det	Avg det/img	Zero-det (imgs)	P95 lat (ms)
n_detect	48	33.7	4.5	556	11.6	0	44.4
s_detect	48	147.9	369.9	679	14.1	0	108.8
n_seg	48	51.3	4.1	660	13.8	0	58.5
s_seg	48	127.5	6.5	712	14.8	0	139.4

ตารางที่ E.1 ผลการ inference บน blurred dataset (48 ภาพ) ด้วย ONNX—latency และจำนวน detection

Model	person	helmet	boots	shoes	harness	Total
n_detect	203	197	35	110	11	556
s_detect	233	244	48	142	12	679
n_seg	272	209	37	123	19	660
s_seg	286	221	58	127	20	712

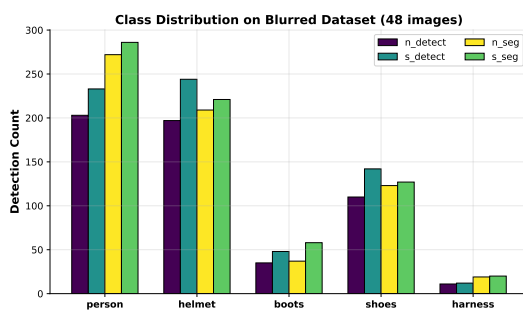
ตารางที่ E.2 Class distribution ของ detection บน blurred dataset (48 ภาพ) ด้วย ONNX

E.3 วิเคราะห์ผล

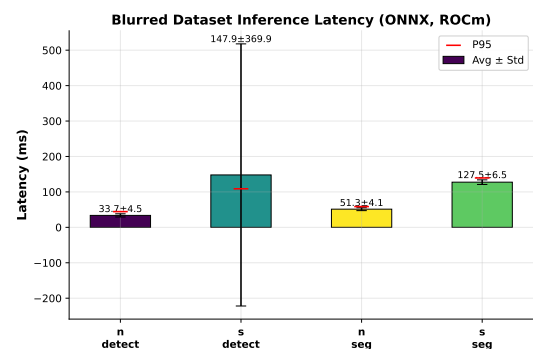
จากตารางที่ E.1 และ E.2:

1. จำนวน detection: s_seg ตรวจจับได้มากที่สุด (712 รายการ, 14.8/img) ตามด้วย s_detect (679, 14.1/img), n_seg (660, 13.8/img), n_detect (556, 11.6/img)
2. Latency: n_detect เร็วที่สุด (33.7 ± 4.5 ms) ส่วน s_detect มี latency สูงและ variance สูง (147.9 ± 369.9 ms) เนื่องจาก outlier จาก ONNX graph initialization ในบางภาพ
3. Zero-detection: ไม่มีภาพใดที่โมเดลตรวจจับไม่ได้เลย (0/48 สำหรับทุก โมเดล) — แสดงว่าโมเดลยัง robust ต่อ blur

4. **Class distribution:** person และ helmet ถูกตรวจจับได้มากที่สุด (203--286 และ 197--244 รายการ) ส่วน harness น้อยที่สุด (11--20 รายการ) — สอดคล้องกับผลบน test set ที่ harness เป็น class ที่ยากที่สุด
5. **Seg vs Detect:** seg models ตรวจจับได้มากกว่า detect models (660--712 vs 556--679) แต่ latency สูงกว่าเนื่องจาก seg ต้องคำนวณ mask prototype เพิ่มเติม



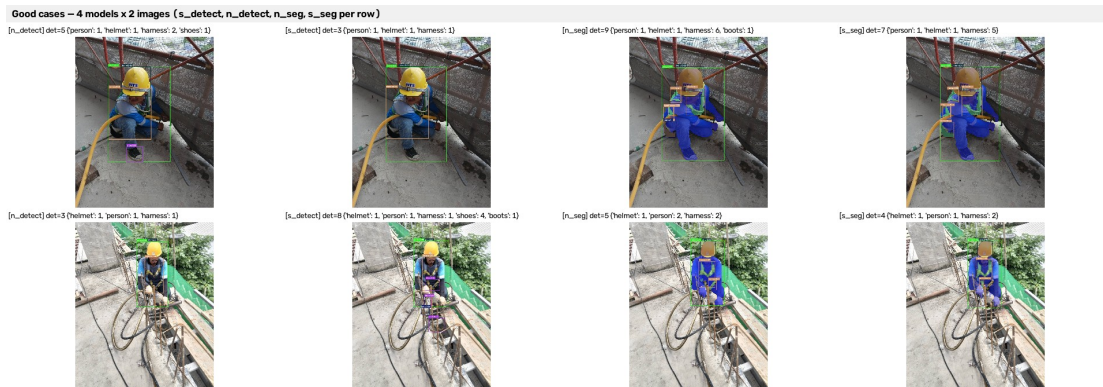
(a) Class distribution ของ detection บน blurred dataset



(b) Latency พร้อม std error bar และ P95 marker

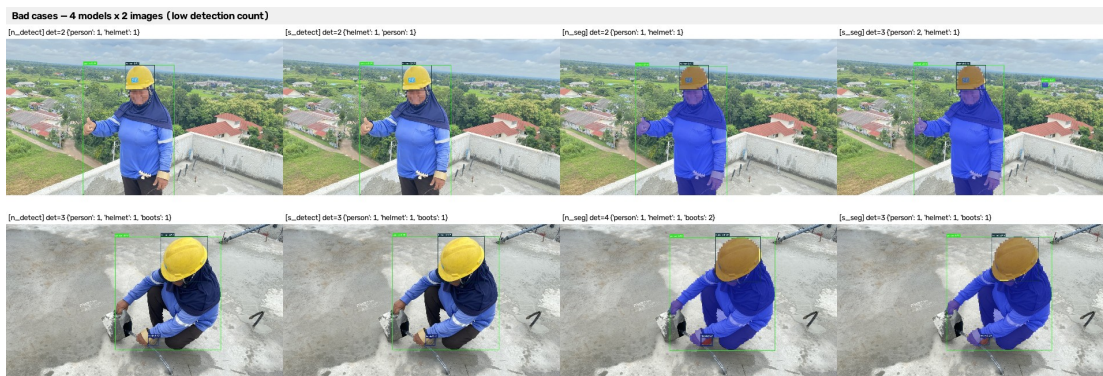
รูปที่ E.1 สถิติการ inference บน blurred dataset (48 ภาพ) ด้วย ONNX: (a) class distribution แสดงว่า person และ helmet ถูกตรวจจับได้มากที่สุด ส่วน harness น้อยสุด (b) latency แสดงว่า n_detect เร็วสุด และ stable ส่วน s_detect มี variance สูง

E.4 Good Cases (หลาย class, หลาย detection)



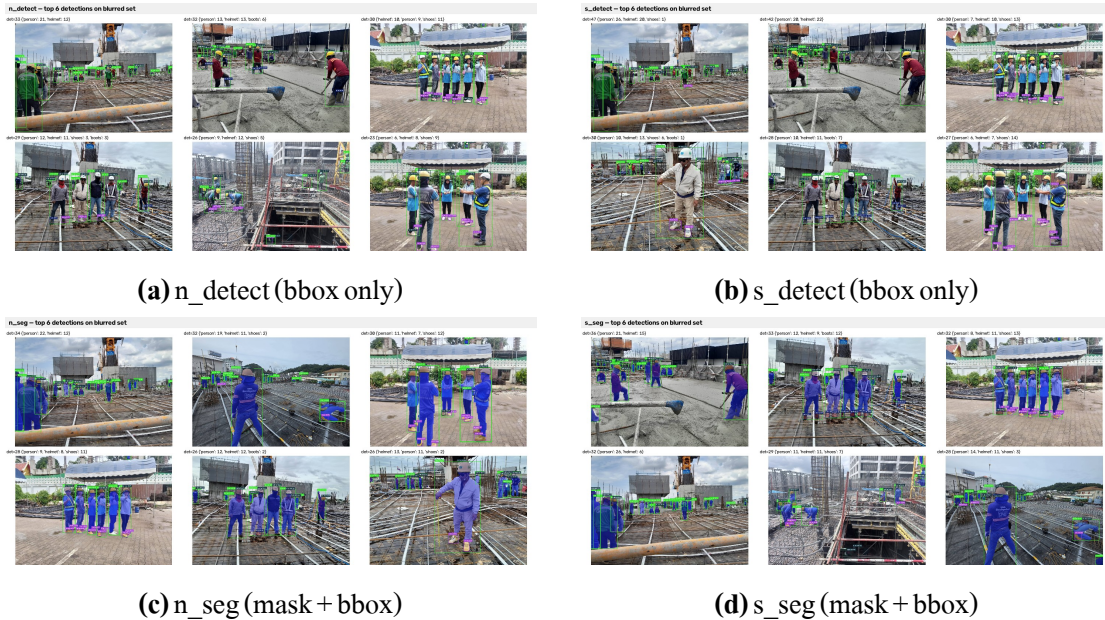
รูปที่ E.2 Good cases บน blurred dataset — 2 ภาพที่ตรวจจับได้ครบทุก class (5 classes) แสดงผล 4 โมเดล side-by-side — กรอบสี = bbox, พื้นหลังสีโปร่ง = mask (seg models) — สีตาม SAM perceptual colormap (LAB kmeans, seed=42) — แถวบน: n_detect, s_detect, n_seg, s_seg สำหรับภาพที่ 1 แถวล่าง: สำหรับภาพที่ 2

E.5 Bad Cases (น้อย class, น้อย detection)



รูปที่ E.3 Bad cases บน blurred dataset — 2 ภาพที่ตรวจจับได้น้อยที่สุด (9--13 รายการ, 2--3 classes) แสดงผล 4 โมเดล side-by-side — สังเกตว่าโมเดลยังตรวจจับ person และ helmet ได้แต่พลาด boots, shoes, harness ในภาพ blur จัด — แสดงว่าโมเดลยังขาด robustness ต่อ blur สำหรับวัตถุขนาดเล็ก

E.6 Per-Model Top Detections



รูปที่ E.4 ภาพที่ตรวจจับได้มากที่สุดบน *blurred dataset* สำหรับแต่ละ โมเดล: (a) n_detect (b) s_detect (c) n_seg (d) s_seg —*detect models* แสดง *bbox* เท่านั้น ส่วน *seg models* แสดง *mask overlay* (สีโปร่ง) พร้อม *bbox*—ติดตาม *class: person, helmet, boots, shoes, harness*

E.7 สรุป Blurred Dataset

- โมเดลทั้ง 4 ตัวยังทำงานได้บนภาพ blurred (0 zero-detection จาก 48 ภาพ)
- s_seg ตรวจจับได้มากที่สุด (712 รายการ) แต่ช้าที่สุด (127.5 ms)
- n_detect เร็วที่สุด (33.7 ms) เหมาะกับ real-time แต่ตรวจจับน้อยสุด (556 รายการ)
- *harness* ยังเป็น *class* ที่ยากที่สุด (11–20 รายการจาก 556–712 รายการทั้งหมด)
- ในภาพ blur จัด โมเดลพลาด *boots, shoes, harness* แต่ยังจับ *person, helmet* ได้
- ข้อเสนอแนะ: ถ้าต้องการ robustness ต่อ blur ควรเพิ่ม augmentation แบบ motion blur + gaussian noise ใน training set